

Microprocessor and Microcontroller

Subject Code:ETEC-305

3rd year ECE

Books to be referred

- **Books:**

[T1] **Muhammad Ali Mazidi, “Microprocessors and Microcontrollers”,
Pearson, 2006**

[T2] B.RAM 8085 programming.

[T3] Douglas V Hall, “Microprocessors and Interfacing, Programming and
Hardware” Tata McGraw Hill, 2006.

Difference B/W

Microprocessor and Microcontroller

It is a central processing unit on a single silicon-based integrated chip.

It has no RAM, ROM, IO units, Timers and other peripherals on the chip.

It is the heart of the computer system.

It uses an external bus to interface to RAM, ROM, IO pins, and other peripherals.

Microprocessor based systems can run at very high speed because of the technology involved.

It's used for general purpose applications which are able to handle loads of data.

It's complex and expensive with large number of instructions to process.

It is a byproduct of the development of microprocessor with a CPU along with other peripherals.

It has a CPU along with RAM, ROM, and other peripherals embedded on a single chip.

It is the brains of the computer system.

It used an internal controlling bus which is not available to the board designer.

Microcontroller based systems run up to 200MHz or more depending on the architecture.

It's used for application specific systems.

It's simple and inexpensive with less number of instructions to process.

UNIT - I

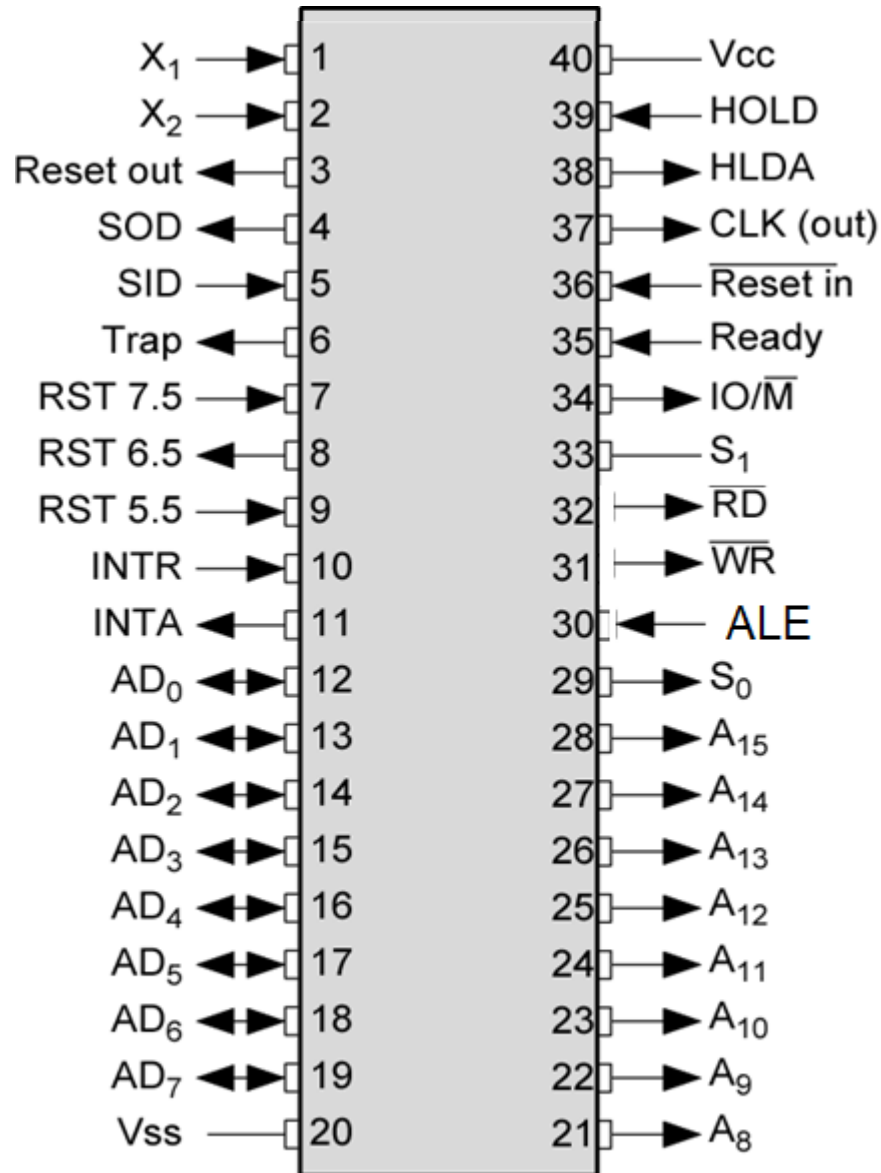
The 8085 Microprocessor

Introduction to Microprocessor Systems: Architecture and PIN diagram of 8085, Timing Diagram, memory organization, Addressing modes, Interrupts. Assembly Language Programming.

The 8085 and Its Busses

- *The 8085 is an **8-bit general purpose** microprocessor that can address $2^{16}=64K$ **Byte of memory**.*
- *It has **40 pins** and uses +5V for power. It can run at a **maximum frequency of 3 MHz**.*
 - *The pins on the chip can be grouped into 6 groups:*
 - ***Address Bus.***
 - ***Data Bus.***
 - ***Control and Status Signals.***
 - ***Power supply***
 - ***Clock signals.***
 - ***Externally Initiated Signals.***
 - ***Serial I/O ports.***

Intel 8085 Pin Configuration



- Maskable interrupt ---
- Non Maskable Interrupt

Address bus

- A15-A8, it carries the most significant 8-bits of memory/IO address.

Data bus

- AD7-AD0, it carries the least significant 8-bit address and data bus.

Control and status signals

These signals are used to identify the nature of operation. There are 3 control signal and 3 status signals.

Three control signals are RD, WR & ALE.

- **RD** – This signal indicates that the selected IO or memory device is to be read and is ready for accepting data available on the data bus.
- **WR** – This signal indicates that the data on the data bus is to be written into a selected memory or IO location.
- **ALE** – It is a positive going pulse generated when a new operation is started by the microprocessor. When the pulse goes high, it indicates address. When the pulse goes down it indicates data.

Three status signals are IO/M, S0 & S1.

IO/M

- This signal is used to differentiate between IO and Memory operations, i.e. when it is high indicates IO operation and when it is low then it indicates memory operation.

S1 & S0

- These signals are used to identify the type of current operation.

Power supply

There are 2 power supply signals – VCC & VSS. VCC indicates +5v power supply and VSS indicates ground signal.

Clock signals

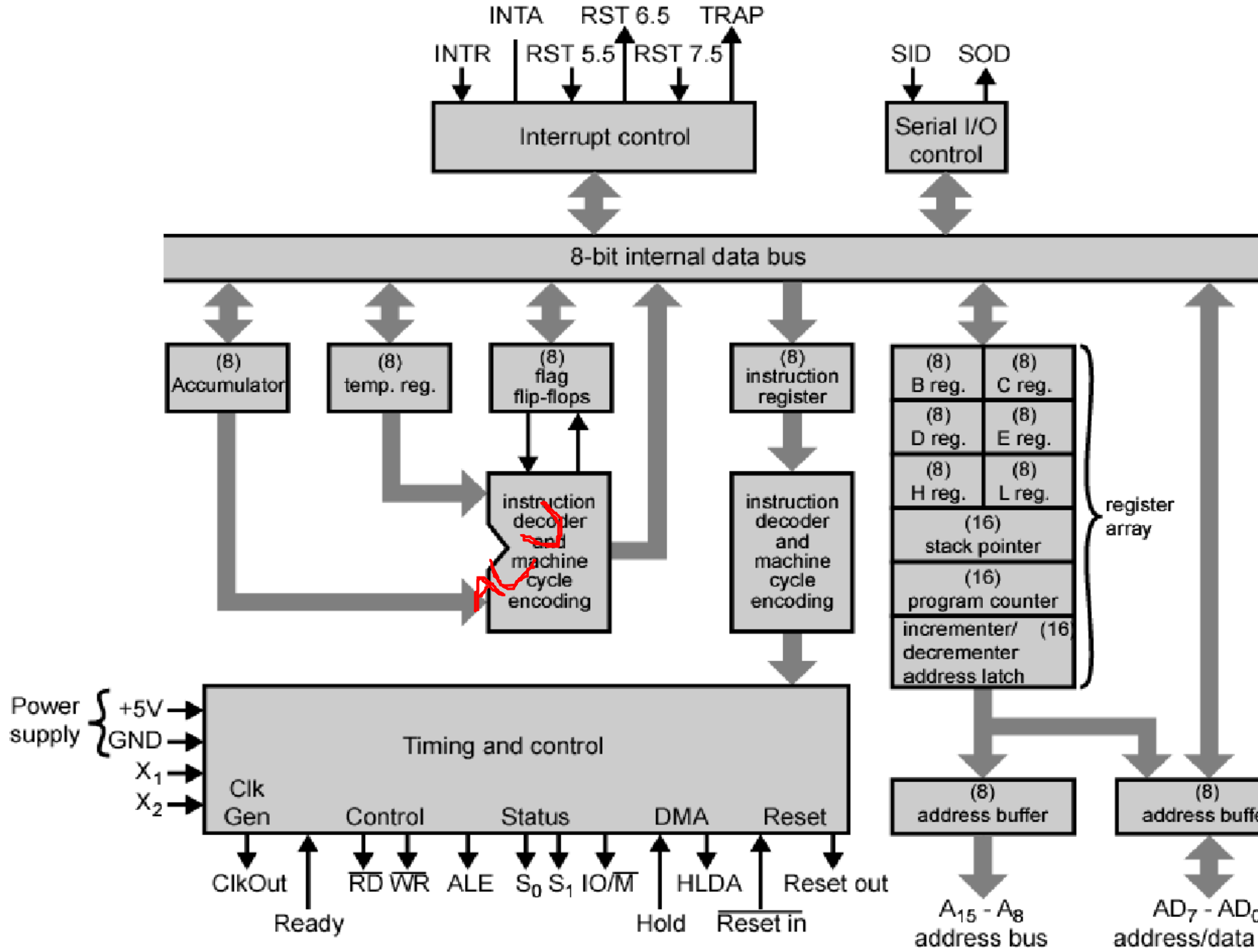
- There are 3 clock signals, i.e. X1, X2, CLK OUT.
- **X1, X2** – A crystal (RC, LC N/W) is connected at these two pins and is used to set frequency of the internal clock generator. This frequency is internally divided by 2.
- **CLK OUT** – This signal is used as the system clock for devices connected with the microprocessor.

Interrupts & externally initiated signals

- Interrupts are the signals generated by external devices to request the microprocessor to perform a task. There are 5 interrupt signals, i.e. TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR. We will discuss interrupts in detail in interrupts section.
- **INTA** – It is an interrupt acknowledgment signal.
- **RESET IN** – This signal is used to reset the microprocessor by setting the program counter to zero.
- **RESET OUT** – This signal is used to reset all the connected devices when the microprocessor is reset.
- **READY** – This signal indicates that the device is ready to send or receive data. If READY is low, then the CPU has to wait for READY to go high.
- **HOLD** – This signal indicates that another master is requesting the use of the address and data buses.
- **HLDA (HOLD Acknowledge)** – It indicates that the CPU has received the HOLD request and it will relinquish the bus in the next clock cycle. HLDA is set to low after the HOLD signal is removed.

Serial I/O signals

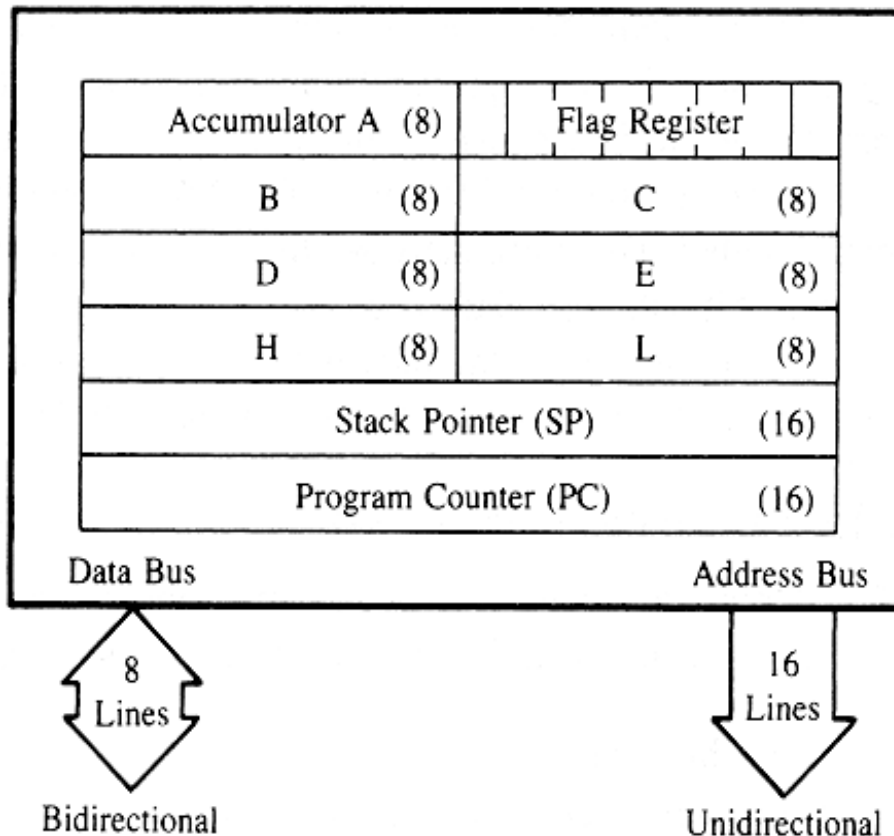
- There are 2 serial signals, i.e. SID and SOD and these signals are used for serial communication.
- **SOD** (Serial output data line) – The output SOD is set/reset as specified by the SIM instruction.
- **SID** (Serial input data line) – The data on this line is loaded into accumulator whenever a RIM instruction is executed.



Functional Blocks

- *Registers*
- *ALU*
- *Instruction Decoder*
- *Address Buffer*
- *Address/Data Buffer*
- *Increment/ Decrement Address Latch*
- *Interrupt Control*
- *Serial I/O Control*
- *Timing and control circuitry*

Registers



- ***General purpose Registers***
- ***Temporary Registers***
- ***Special Purpose Register***
- ***16 Bit Registers***

The Flags register

- There is also the flags register whose bits are affected by the arithmetic & logic operations.

- **S-sign flag**

- The sign flag is set if bit D7 of the accumulator is set after an arithmetic or logic operation.

– 0- +Ve 1- -Ve

- **Z-zero flag**

- Set if the result of the ALU operation is 0. Otherwise is reset. This flag is affected by operations on the accumulator as well as other registers. (DCR B).

- **AC-Auxiliary Carry**

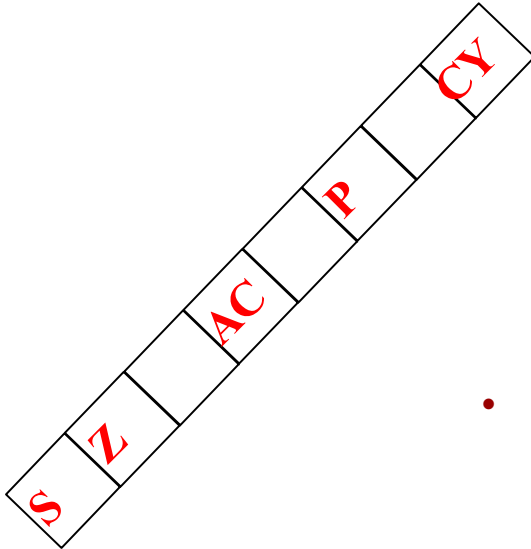
- This flag is set when a carry is generated from bit D3 and passed to D4 . This flag is used only internally for BCD operations.

- **P-Parity flag**

- After an ALU operation if the result has an even no of 1's the p- flag is set. Otherwise it is cleared. So, the flag can be used to indicate even parity.

- **CY-carry flag**

- CY = carry is set when result generates a carry. Also a borrow flag.



PROGRAM COUNTER (PC) AND STACK POINTER (SP)

- **These are two 16-bit registers used to hold memory addresses.**
- **PC:**
 - **The function of the PC is to point to the memory address from which the next byte is to be fetched.**
 - **When a byte (machine code) is being fetched, the program counter is incremented by one to point to the next memory location.**
- **SP:**
 - **It points to a memory location in R/W memory, called the stack.**
 - **The beginning of the stack is defined by loading a 16-bit address in the stack pointer.**
 - **The PC will automatically update when calling to /returning from Subroutines.**

The ALU

- *In addition to the arithmetic & logic circuits, the ALU includes the accumulator, which is part of every arithmetic & logic operation.*
- *Also, the ALU includes a temporary register used for holding data temporarily during the execution of the operation. This temporary register is not accessible by the programmer.*

The Address and Data Busses

- The address bus has 8 signal lines **A8 – A15** which are **unidirectional**.
- The other 8 address bits are **multiplexed** (time shared) **with the 8 data bits**.
 - So, the bits **AD0 – AD7** are **bi-directional** and serve as **A0 – A7** and **D0 – D7** at the same time.
 - During the execution of the instruction, these lines carry the address bits during the early part, then during the late parts of the execution, they carry the 8 data bits.
 - In order to separate the address from the data, we can use a latch to save the value before the function of the bits changes.

INCREMENT/ DECREMENT ADDRESS LATCH

Interrupt Control:

Fetch, Decode & execute

RST 5.5, RST 6.5, RST 7.5, TRAP, & INTR

Serial I/O Control

SOD & SID

Timing and Control Circuitry

Its control fetching, decoding, execution

Frequency Control Signals

- *There are **3** important pins in the frequency control group.*
 - ***X0** and **X1** are the **inputs** from the **crystal** or clock generating circuit.*
 - ***The frequency is internally divided by 2.***
 - *So, to run the microprocessor at 3 MHz, a clock running at 6 MHz should be connected to the X0 and X1 pins.*
 - ***CLK** (**OUT**): An output clock pin to drive the clock of the rest of the system.*
- *We will discuss the rest of the control signals as we get to them.*

The Control and Status Signals

- There are **4** main **control** and **status** signals. These are:
 - **ALE: Address Latch Enable**. This signal is a pulse that become **1** when the **AD0 – AD7** lines have an **address** on them. It becomes 0 after that. This signal can be used to enable a latch to save the address bits from the AD lines.
 - **RD: Read. Active low** indicates that the data must be read from the selected memory location or I/O port via data bus.
 - **WR: Write. Active low** indicates that the data must be written into the selected memory location or I/O port via data bus..
 - **IO/M**: This signal specifies whether the operation is a **memory operation** (**IO/M=0**) or an **I/O operation** (**IO/M=1**).
 - **S1 and S0** : Status signals to specify the **kind of operation** being performed .Usually un-used in small systems.

These can be guided by addressing modes. Addressing modes in 8085 can be classified into 5 groups –

Immediate addressing mode

Register addressing mode

Direct addressing mode

Indirect addressing mode

Implied addressing mode

ADDRESSING MODES

- **IMMEDIATE**

MVI A,05(H) LXI H,0050(H)

- **DIRECT**

LDA 0208(H)

- **REGISTER**

MOV B,C

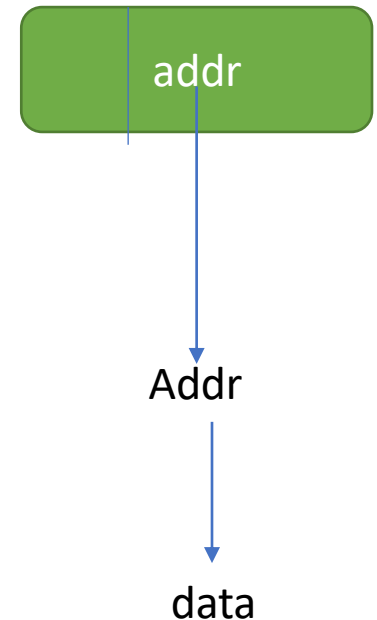
ADD B

- **REGISTER INDIRECT**

LDAX B

- **IMPLIED**

HLT STC(SET CARRY FLAG)



Immediate addressing mode

In this mode, the 8/16-bit data is specified in the instruction itself as one of its operands. For example MVI E, ABH means ABH is copied into register A.

▣ MVI E ABH

	Before	After
(A)	Any value	ABH

As an example, if we consider instruction **MVI E, ABH** then it means that ABH will be moved or copied to the register E.And, as a result, the previous value of E will get overwritten.

Address	Hex Codes	Mnemonic	Comment
2000	1E	MVI E, ABH	$E \leftarrow ABH$
2001	AB		ABH as operand

Register addressing mode

In this mode, the data is copied from one register to another. For example, MOV A, B: means data in register B is copied to register A.

- MOV E, H

It occupies only 1-Byte in memory. **MOV E, H** is an example instruction of this type. It is a 1-Byte instruction. Suppose E register content is AB H, and H register content is 9C H. When the 8085 executes this instruction, the contents of E register will change to 9C H. This is shown as follows.

	Before	After
(E)	ABH	9CH
(H)	9CH	9CH

NOTATIONS	MEANING
M	Memory location pointed by HL reg pair
r	8- bit register
rp	16-bit register
rs	Source register
rd	Destination register
addr	16-bit address

Data Transfer Group

- *MVI r, data (8- Bit)*
- *MVI M, data (8- Bit)*
- *MOV rd, rs*
- *MOV M, rs*
- *MOV rd, M*
- *LXI rp, data (16 bit)*
- *STA addr*
- *LDA addr*
- *SHLD addr*
- *LHLD addr*
- *STAX addr*
- *LDAX rp*
- *XCHG*

- **1.MOV:** - This instruction is used to copy the data from one place to another.

Eg: - MOV Rd, Rs (This instruction copies the content of Rs to Rd)

MOV M,Rs (This instruction copies the content of register Rs to memory location pointed by HL Register)

MOV Rd, M (This instruction copies the content of memory location pointed by the HL register to the register Rd.)

2.MVI: - move immediate data to a register or memory location.

Eg: - MVI Rd, #30H (30h is stored in register Rd)

MVI M, #30H(30h is stored in memory location pointed by HL Reg)

3.LDA: - Load accumulator. (this instruction copies the data from a given 16 bit address to the accumulator)

Eg: - LDA 3000H (content of memory location 3000h is copied in accumulator)

4.LDAX(Load accumulator indirect): - The contents of the designated register pair point to a memory location. This instruction copies the contents of that memory location into the accumulator. The contents of either the register pair or the memory location are not altered.

Eg: - LDAX B

5.LXI(Load register pair immediate): - The instruction loads 16-bit data in the register pair designated in the operand.

Eg: - LXI H, 2034H (2034H is stored in HL pair so that it act as memory pointer)

LXI H, XYZ (address of level XYZ is copied in HL pair)

6.LHLD(Load H and L register direct): - this instruction loads the contents of the 16-bit memory location into the HL register pair.

Eg: - LHLD 3000H (the content of location 3000h is copied into the HL reg pair)

- *MOV Rd, Rs* *MOV B, C*
- *MVI R, 8bit* *MVI A, 05H*
- *LXI Rp, 16bit* *LXI B, 2050*
- *MOV R, M* *MOV D, M*
- *MOV M, R* *MOV M, E*
- *LDA 16 bit* *LDA 8005H*
- *STA 16 bit* *STA 8006H*
- *LDAx Rp* *LDAx B*
- *STAX Rp* *STAX D*

ARITHMETIC GROUP

- *ADD r*
- *ADD M*
- *ADI data (8)*
- *ADC r*
- *ADC M*
- *ACI data (8)*
- *DAD rp*
- *SUB r*
- *SUB M*
- *SUI data*
- *SBB r*
- *SBB M*
- *SBI data*
- *DAA*
- *INR r*
- *INR M*
- *INX rp*
- *DCR r*
- *DCR M*
- *DCX rp*

ADD	R M	Add register or memory, to the accumulator	The contents of the register or memory are added to the contents of the accumulator and the result is stored in the accumulator. Example – ADD K.
ADC	R M	Add register to the accumulator with carry	The contents of the register or memory & M the Carry flag are added to the contents of the accumulator and the result is stored in the accumulator. Example – ADC K
ADI	8-bit data	Add the immediate to the accumulator	The 8-bit data is added to the contents of the accumulator and the result is stored in the accumulator. Example – ADI 55K
ACI	8-bit data	Add the immediate to the accumulator with carry	The 8-bit data and the Carry flag are added to the contents of the accumulator and the result is stored in the accumulator. Example – ACI 55K

DAD	Reg. pair	Add the register pair to H and L registers	The 16-bit data of the specified register pair are added to the contents of the HL register. Example – DAD K
SUB	R M	Subtract the register or the memory from the accumulator	The contents of the register or the memory are subtracted from the contents of the accumulator, and the result is stored in the accumulator. Example – SUB K
SBB	R M	Subtract the source and borrow from the accumulator	The contents of the register or the memory & M the Borrow flag are subtracted from the contents of the accumulator and the result is placed in the accumulator. Example – SBB K
SUI	8-bit data	Subtract the immediate from the accumulator	The 8-bit data is subtracted from the contents of the accumulator & the result is stored in the accumulator. Example – SUI 55K
SBI	8-bit data	Subtract the immediate from the accumulator with borrow	The contents of register H are exchanged with the contents of register D, and the contents of register L are exchanged with the contents of register E. Example – XCHG

- *ADD R*
- *ADI 8bit*
- *SUB R*
- *SUI 8bit*
- *INR R*
- *DCR R*
- *INR M*
- *DCR M*
- *INX Rp*
- *DCX Rp*

ADD B
ADI 59H
SUB C
SUI 37H
INR D
DCR B
INR M
DCRB M
INX B

LOGIC GROUP

- ***ANA r***
- ***ANAM***
- ***ANI data***
- ***XRA r***
- ***XRAM***
- ***XRI data***
- ***ORA r***
- ***ORAM***
- ***ORI data***
- ***CMP r***
- ***CMP M***
- ***CPI data***
- ***STC***
- ***CMC***
- ***CMA***

CMP	R M	Compare the register or memory with the accumulator	The contents of the operand (register or memory) are M compared with the contents of the accumulator.
CPI	8-bit data	Compare immediate with the accumulator	The second byte data is compared with the contents of the accumulator.
ANA	R M	Logical AND register or memory with the accumulator	The contents of the accumulator are logically AND with M the contents of the register or memory, and the result is placed in the accumulator.
ANI	8-bit data	Logical AND immediate with the accumulator	The contents of the accumulator are logically AND with the 8-bit data and the result is placed in the accumulator.
XRA	R M	Exclusive OR register or memory with the accumulator	The contents of the accumulator are Exclusive OR with M the contents of the register or memory, and the result is placed in the accumulator.
XRI	8-bit data	Exclusive OR immediate with the	The contents of the accumulator are Exclusive OR with the 8-bit data and the result is placed in the accumulator.

ORA	R M	Logical OR register or memory with the accumulator	The contents of the accumulator are logically OR with M the contents of the register or memory, and result is placed in the accumulator.
ORI	8-bit data	Logical OR immediate with the accumulator	The contents of the accumulator are logically OR with the 8-bit data and the result is placed in the accumulator.
RLC	None	Rotate the accumulator left	Each binary bit of the accumulator is rotated left by one position. Bit D7 is placed in the position of D0 as well as in the Carry flag. CY is modified according to bit D7.
RRC	None	Rotate the accumulator right	Each binary bit of the accumulator is rotated right by one position. Bit D0 is placed in the position of D7 as well as in the Carry flag. CY is modified according to bit D0.
RAL	None	Rotate the accumulator left through carry	Each binary bit of the accumulator is rotated left by one position through the Carry flag. Bit D7 is placed in the Carry flag, and the Carry flag is placed in the least significant position D0. CY is modified according to bit D7.
RAR	None	Rotate the accumulator right through carry	Each binary bit of the accumulator is rotated right by one position through the Carry flag. Bit D0 is placed in the Carry flag, and the Carry flag is placed in the most significant position D7. CY is modified according to bit D0.

- *ANA R/M*

ANAD

- *ANI 8bit*

- *ORA R/M*

ORAC

- *ORI 8bit*

- *XRA R/M*

XRAD

- *XRI 8bit*

- *RLC*

- *RAL*

- *RRC*

- *RAR*

- *CMP R/M*

BRANCH GROUP

- *JUMP
INSTRUCTION*
- *CALL and
RETURN
INSTRUCTION*
- *RESTART
INSTRUCTION*

- ***JMP 16bitAddress***
 - ***JZ 16bit Address***
 - ***JNZ 16bit Address***
 - ***JC 16bit Address***
 - ***JNC 16bit Address***
-
- ***HLT*** ***Stop processing and wait***
 - ***NOP*** ***Don't perform anything - delay***

STACK OPERATION I/O & *CONTROL GROUP*

- *PUSH*
 - *POP*
 - *RESTART*
- RST n*

INSTRUCTION FORMATS

```
graph TD; A[INSTRUCTION FORMATS] --> B[1 BYTE INSTRUCTION]; A --> C[2 BYTE INSTRUCTION]; A --> D[3 BYTE INSTRUCTION]; B --> E["MOV A,B --- 78H"]; C --> F["MVI B, 02 --- 06H 02"]; D --> G["JMP 6200H --- C3H 00 62"];
```

1 BYTE INSTRUCTION

MOV A,B --- 78H

2 BYTE INSTRUCTION

MVI B, 02 --- 06H 02

3 BYTE INSTRUCTION

JMP 6200H ---
C3H 00 62

INSTRUCTION SET

INSTRUCTION SET

```
graph TD; IS[INSTRUCTION SET] --> MI[MOVEMENT INSTUCTIONS]; IS --> MOI[MODIFICATION INSTUCTIONS]; IS --> CI[CONTROL INSTRUCTIONS]; MI --> G0[GROUP-0 DATA TRANSFER]; MI --> G1[GROUP-1 DATA TRANSFER]; G0 --> G0_L["MVI, INR, DCR, LDA, STA, RAR, CMC, CMA, STC, DAA, DAD, LDAX, SHLD, INX, RIM ETC.,"]; G1 --> MOV[MOV]; MOI --> G2[GROUP-2 ARITHMETATIC & LOGIC]; G2 --> G2_L["AND, ADD, OR, XOR etc.,"]; CI --> G3A[GROUP-3A BRANCH]; CI --> G3B[GROUP-3B I/O & MACHINE CONTROL]; G3A --> G3A_L["JNZ, JNC, JC, JZ etc., PROGRAM CONTROL"]; G3B --> G3B_L["HLT, ENABLE, DISABLE, INTR PROCESS CONTROL"];
```

MOVEMENT INSTUCTIONS

GROUP-0 DATA TRANSFER

MVI, INR, DCR, LDA, STA, RAR, CMC, CMA, STC, DAA, DAD, LDAX, SHLD, INX, RIM ETC.,

GROUP-1 DATA TRANSFER

MOV

MODIFICATION INSTUCTIONS

GROUP-2 ARITHMETATIC & LOGIC

AND, ADD, OR, XOR etc.,

CONTROL INSTRUCTIONS

GROUP-3A BRANCH

JNZ, JNC, JC, JZ etc.,
PROGRAM CONTROL

GROUP-3B I/O & MACHINE CONTROL

HLT, ENABLE, DISABLE, INTR
PROCESS CONTROL

BYTE ORGANIZATION

GROUP - 0

0	0	R	R	R	I0	I0	I0
---	---	---	---	---	----	----	----

GROUP - 1

0	1	R	R	R	S	S	S
---	---	---	---	---	---	---	---

GROUP - 2

1	0	A1	A1	A1	S	S	S
---	---	----	----	----	---	---	---

GROUP - 3

1	1	Cb	Cb	Cb	B0	B0	B0
---	---	----	----	----	----	----	----

CODE FOR RECEIVING AND SENDING *REGISTERS/PAIRS*

REGISTERS	ADDRESS CODE	REGISTERS	ADDRESS CODE
B	000	B – C	00
C	001		
D	010		
E	011	D – E	01
H	100		
L	101		
M	110	H – L	10
A	111		
		SP	11

INFORMATION OPERATIONS (I0 I0 I0)

ADDRESS			OPERATION
I0	I0	I0	
0	0	0	NOT USED
0	0	1	IMMETIATE OPERATION REGISTER PAIR
0	1	0	LOAD / STORE
0	1	1	INCREMENT/ DECREMENT REGISTER PAIR
1	0	0	INCREMENT SINGLE REGISTER
1	0	1	DECREMENT SINGLE REGISTER
1	1	0	IMMETIATE OPERATION ON SINGLE REGISTER
1	1	1	REGISTER SHIFTING

ARITHMETIC AND LOGICAL OPERATIONS (A1 A1 A1)

ADDRESS			OPERATION
A1	A1	A1	
0	0	0	ADD
0	0	1	ADD WITH CARRY (ADC)
0	1	0	SUBTRACT (SUB)
0	1	1	SUBTRACT WITH BORROW (SBB)
1	0	0	LOGICAL AND
1	0	1	EXCLUSIVE OR (X-OR)
1	1	0	LOGICAL OR (OR)
1	1	1	COMPARE

CONDITIONS OF BRANCH (Cb Cb Cb)

ADDRESS			OPERATION
Cb	Cb	Cb	
0	0	0	IF NOT ZERO (JNZ)
0	0	1	IF ZERO (JZ)
0	1	0	IF NO CARRY(JNC)
0	1	1	IF CARRY (JC)
1	0	0	IF ODD PARITY (JPO)
1	0	1	IF EVEN PARITY (JPE)
1	1	0	WAS IT POSITIVE (JP)
1	1	1	WAS IT NEGATIVE (JM)

BRANCH OPERATIONS (B0 B0 B0)

ADDRESS			OPERATION
B0	B0	B0	
0	0	0	CONDITIONAL RETURN
0	0	1	SIMPLE RETURN
0	1	0	CONDITIONAL JUMP
0	1	1	UNCONDITIONAL JUMP
1	0	0	CONDITIONAL CALL
1	0	1	SIMPLE CALL
1	1	0	SPECIAL A/L OPERATIONS
1	1	1	SPECIAL UNCONDITIONAL JUMPS

MVI B, BYTE

0	0	R				I0	I0	I0
---	---	---	--	--	--	----	----	----

0	0	0	0	0		1	1	0
---	---	---	---	---	--	---	---	---

MOV B,C

0	1	R	R	R		S	S	S
---	---	---	---	---	--	---	---	---

0	1	0	0	0		0	0	1
---	---	---	---	---	--	---	---	---

ADD B

1	0	A1	A1	A1		S	S	S
---	---	----	----	----	--	---	---	---

1	0	0	0	0		0	0	0
---	---	---	---	---	--	---	---	---

Static RAM	Dynamic RAM
Static RAM contains less memory cells per unit area	Dynamic RAM contains more memory cells as compare to static RAM per unit area
It has less access time, hence faster memories	Its access time is greater than static RAMs
Static RAM consists of number of flip-flops. Each flip-flop stores one bit	Dynamic RAM stores the data as a charge on the capacitor. It consists of MOSFET and the capacitor for each cell
Refreshing circuitry is not required	Refreshing circuitry is required to maintain the charge on the capacitor after every few milliseconds
Cost is more	Cost is less

MACHINE CYCLES AND THEIR TIMING OF 8085:

Timing Diagram is a graphical representation. It represents the execution time taken by each instruction in a graphical format. The execution time is represented in T-states.

Instruction Cycle:

The time required to execute an instruction is called instruction cycle.

Machine Cycle:

The time required to access the memory or input/output devices is called machine cycle.

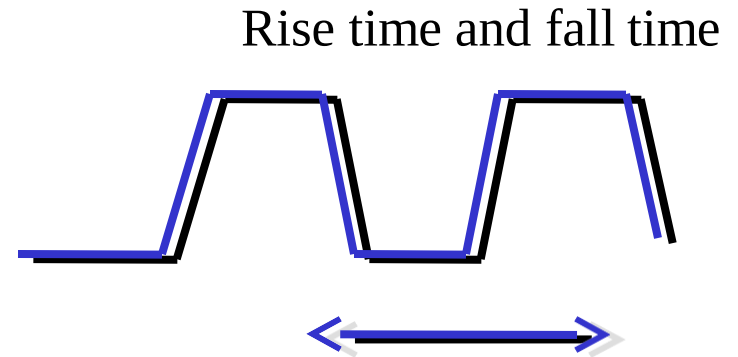
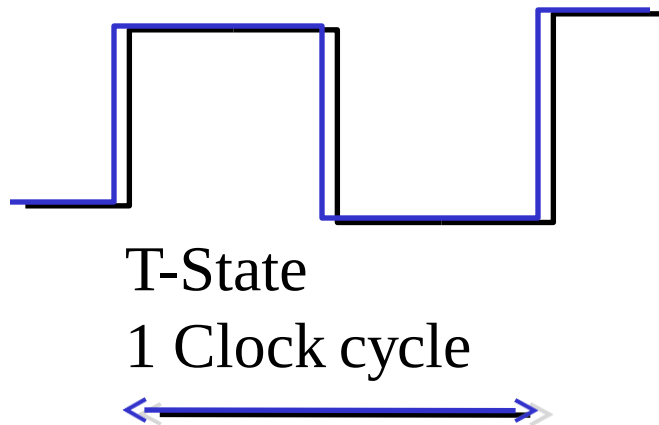
T-State:

The machine cycle and instruction cycle takes multiple clock periods.

A portion of an operation carried out in one system clock period is called as T-state.

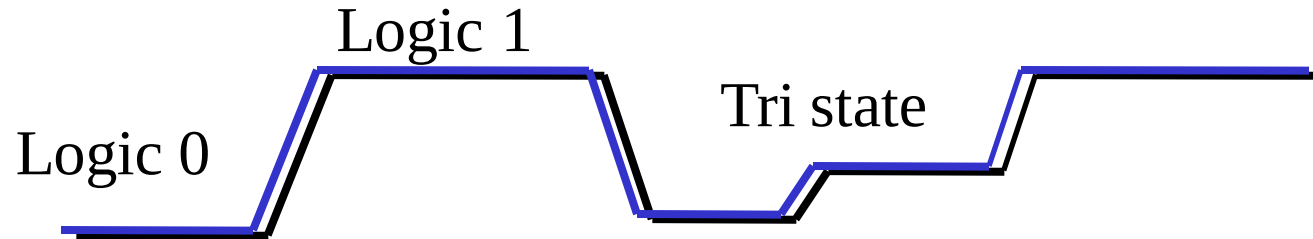
- ***Clock Signal***

The 8085 divides the clock frequency provided at x1 and x2 inputs by 2 which is called operating frequency.

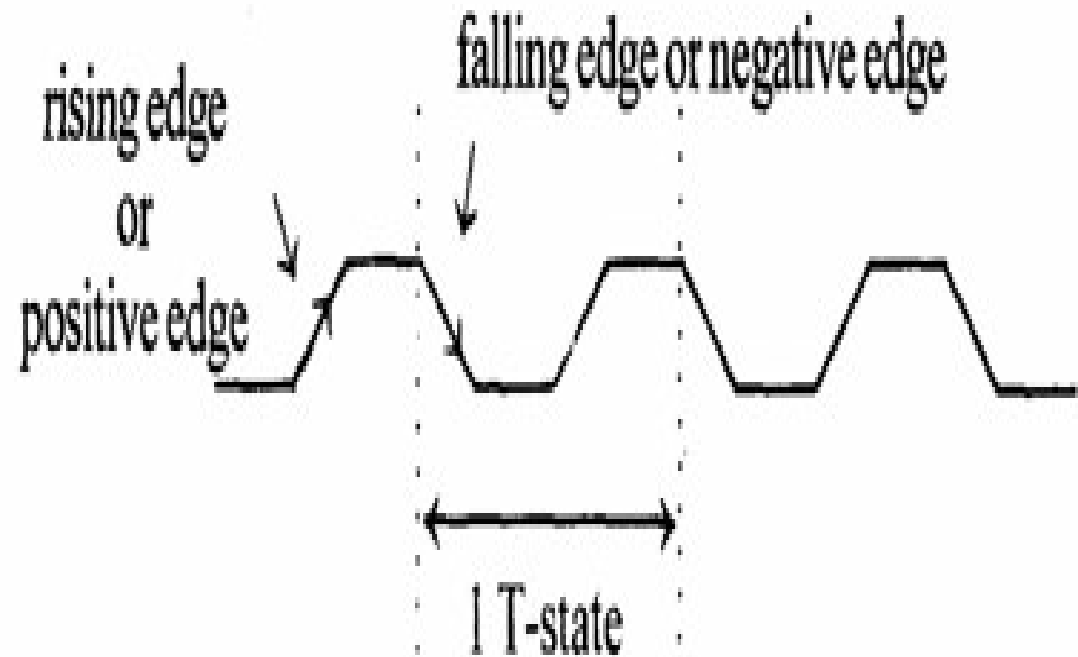


- ***Single Signal***

Single signal status is represented by a line. It may have status either logic 0 or logic 1 or tri-state



Note : Time period, $T = 1/f$; where f = Internal clock frequency



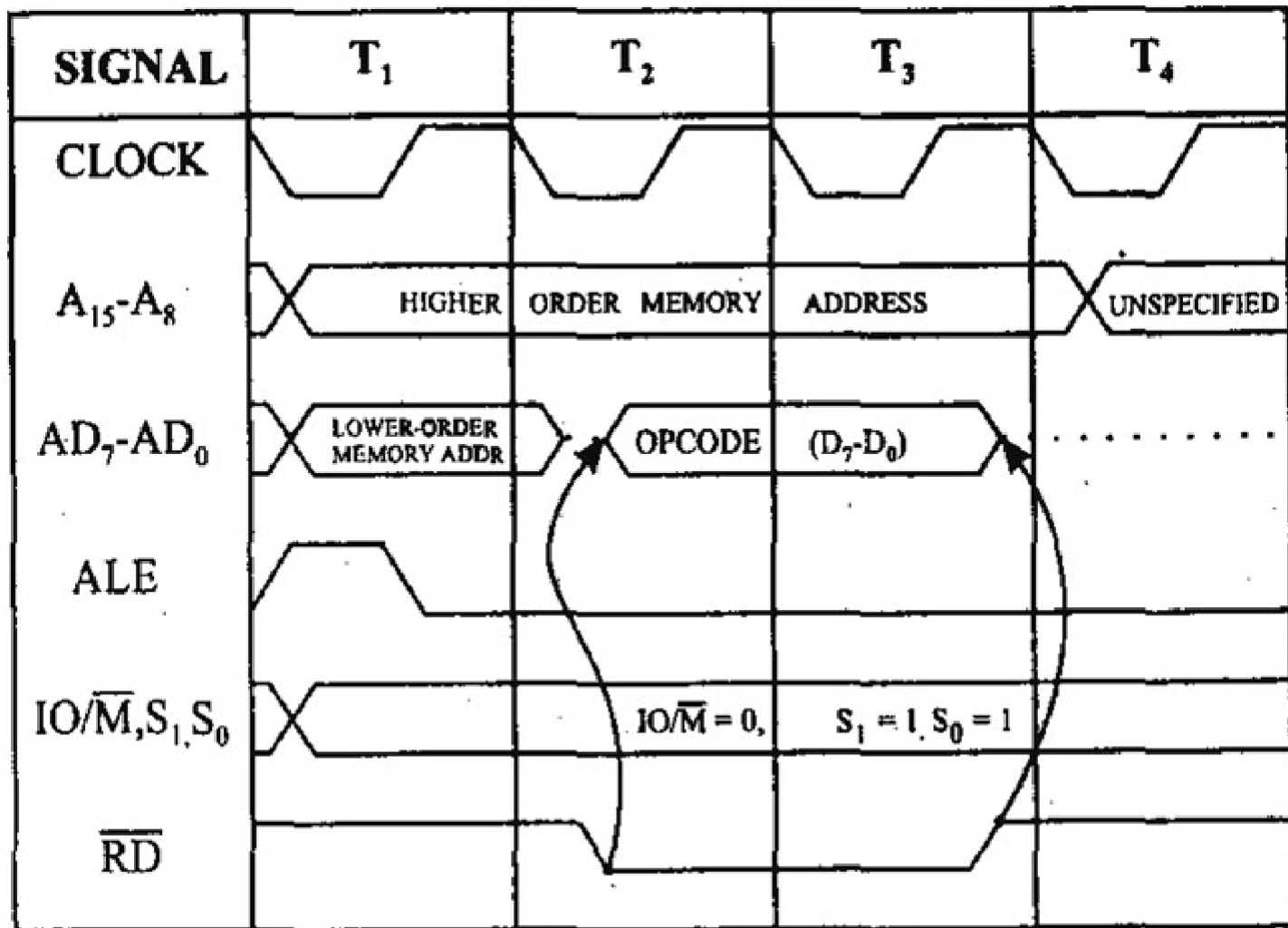
The 8085 microprocessor has 5 basic machine cycles. They are

1. Opcode fetch cycle (4T)
2. Memory write cycle (3 T)
3. I/O read cycle (3 T)
4. Memory read cycle (3 T)
5. I/O write cycle (3 T)

- Each instruction of the 8085 processor consists of one to five machine cycles, i.e., when the 8085 processor executes an instruction, it will execute some of the machine cycles in a specific order.
- The processor takes a definite time to execute the machine cycles. The time taken by the processor to execute a machine cycle is expressed in T-states.
- One T-state is equal to the time period of the internal clock signal of the processor.
- The T-state starts at the falling edge of a clock.
- In this time, the first, 3 T-states are used for fetching the opcode from memory and the remaining T-states are used for internal operations by the processor.

Opcode Fetch Machine Cycle

- *The first step of executing any instruction is the **Opcode fetch cycle**.*
 - *In this cycle, the microprocessor brings in the instruction's Opcode from memory.*
 - *To differentiate this machine cycle from the very similar “memory read” cycle, the control & status signals are set as follows:*
 - ***IO/M=0**, **s0** and **s1** are both **1**.*
 - *This machine cycle has four T-states.*
 - *The 8085 uses the first 3 T-states to fetch the opcode.*
 - *T4 is used to decode and execute it.*
 - *It is also possible for an instruction to have 6 T-states in an opcode fetch machine cycle.*



Timing Diagram for Opcode Fetch Machine Cycle

Memory Read Machine Cycle of 8085:

- The memory read machine cycle is executed by the processor to read a data byte from memory.
- The processor takes 3T states to execute this cycle.
- The instructions which have more than one byte word size will use the machine cycle after the opcode fetch machine cycle.

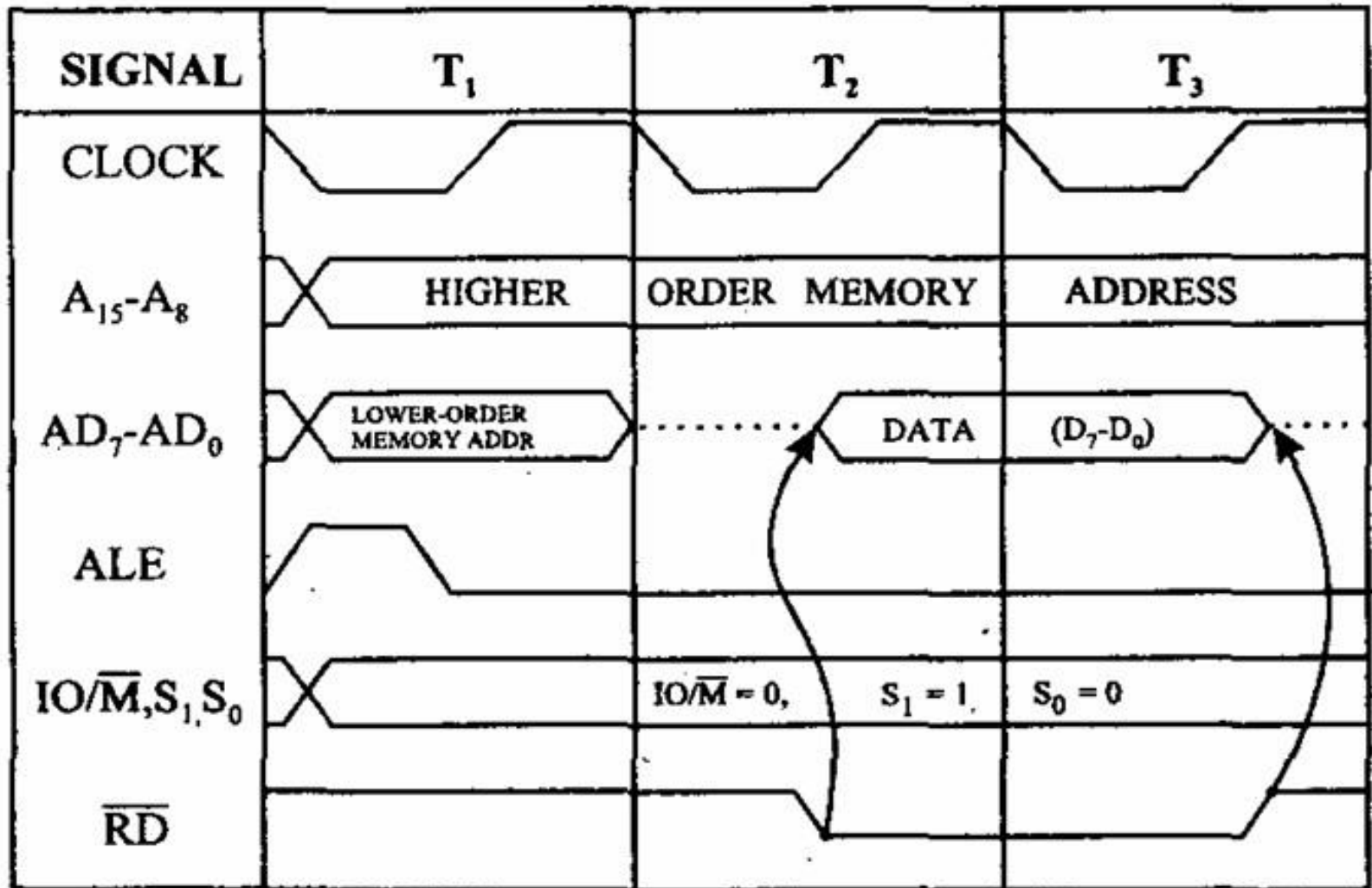
Memory Read Machine Cycle

- *The memory read machine cycle is exactly the same as the opcode fetch except:*
 - *It only has 3 T-states*
 - *The **s0** signal is set to **0** instead.*

The Memory Read Machine Cycle

- *To understand the memory read machine cycle, let's study the execution of the following instruction:*
 - *MVI A, 32*
- *In memory, this instruction looks like:*
 - *The first byte 3EH represents the opcode for loading a byte into the accumulator (MVI A), the second byte is the data to be loaded.*
- *The 8085 needs to read these two bytes from memory before it can execute the instruction. Therefore, it will need at least two machine cycles.*
 - *The first machine cycle is the opcode fetch discussed earlier.*
 - *The second machine cycle is the Memory Read Cycle*

2000H	3E
2001H	32



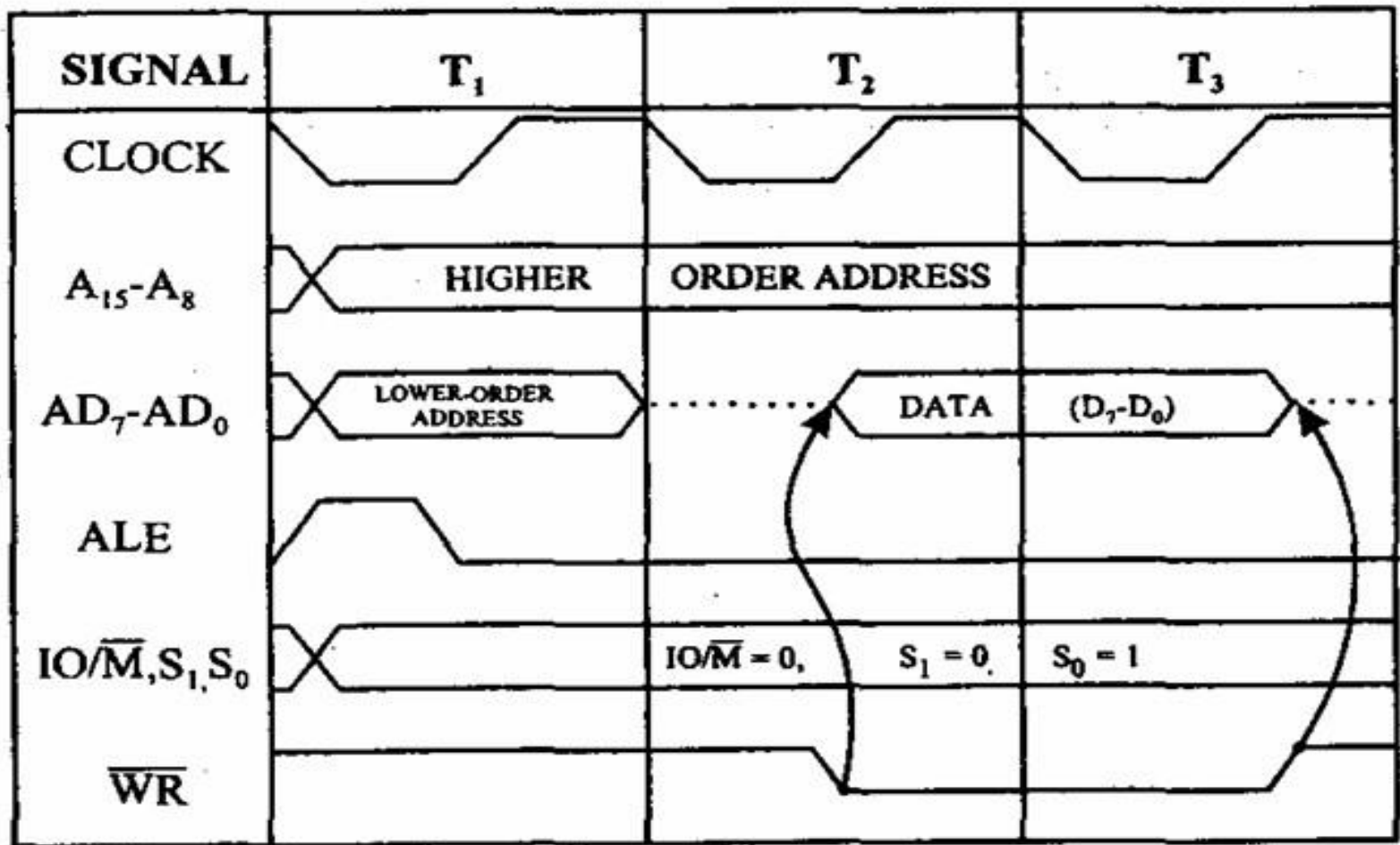
Timing Diagram for Memory Read Machine Cycle

Memory Write Machine Cycle of 8085:

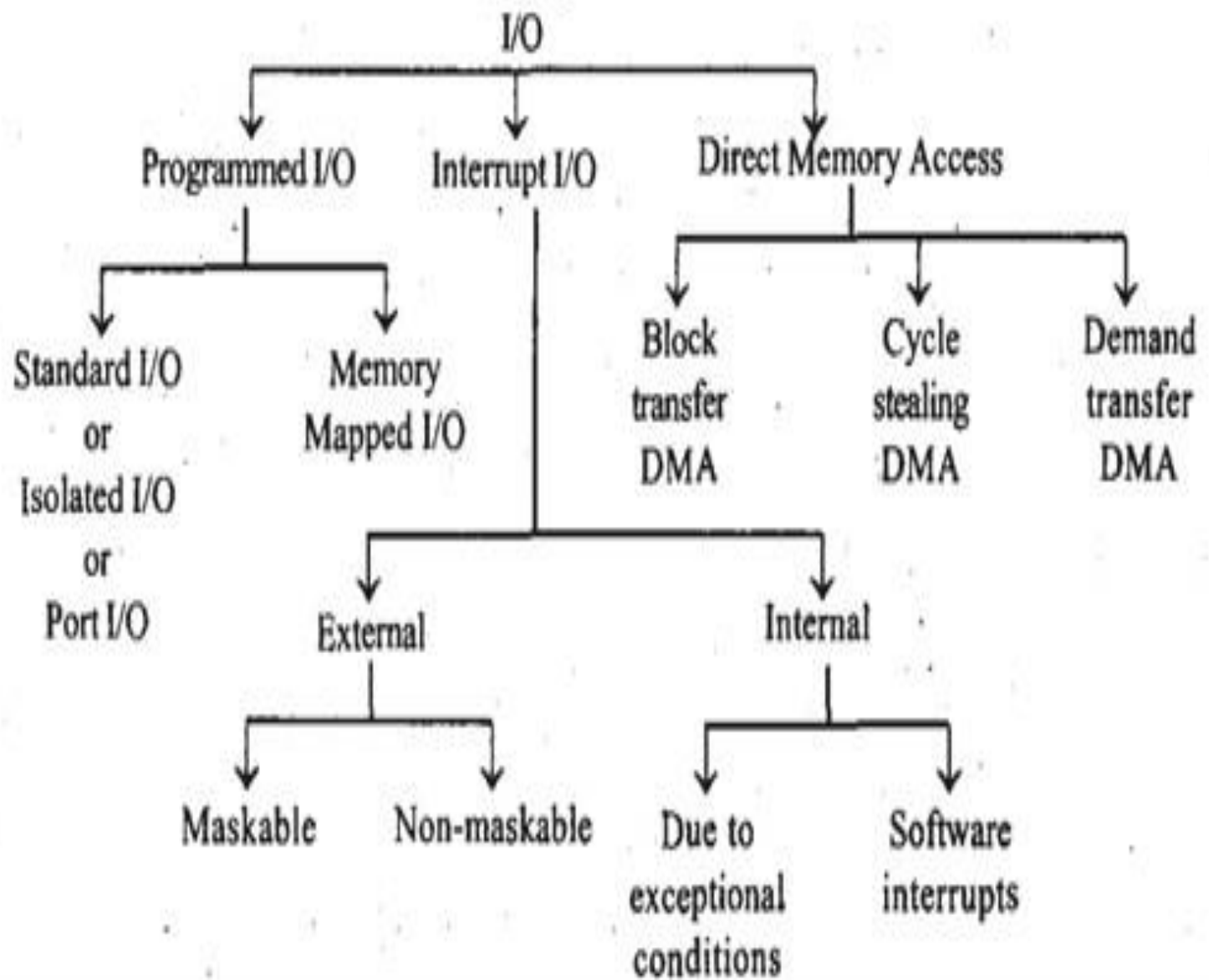
- The memory write machine cycle is executed by the processor to write a data byte in a memory location.
- The processor takes, 3T states to execute this machine cycle.

The Memory Write Operation

- ***In a memory write operation:***
 - ***The 8085 places the address (2065H) on the address bus***
 - ***Identifies the operation as a memory write (IO/M=0, s1=0, s0=1).***
 - ***Places the contents of the accumulator on the data bus and asserts the signal WR.***
 - ***During the last T-state, the contents of the data bus are saved into the memory location.***



Timing Diagram for Memory Write Machine Cycle

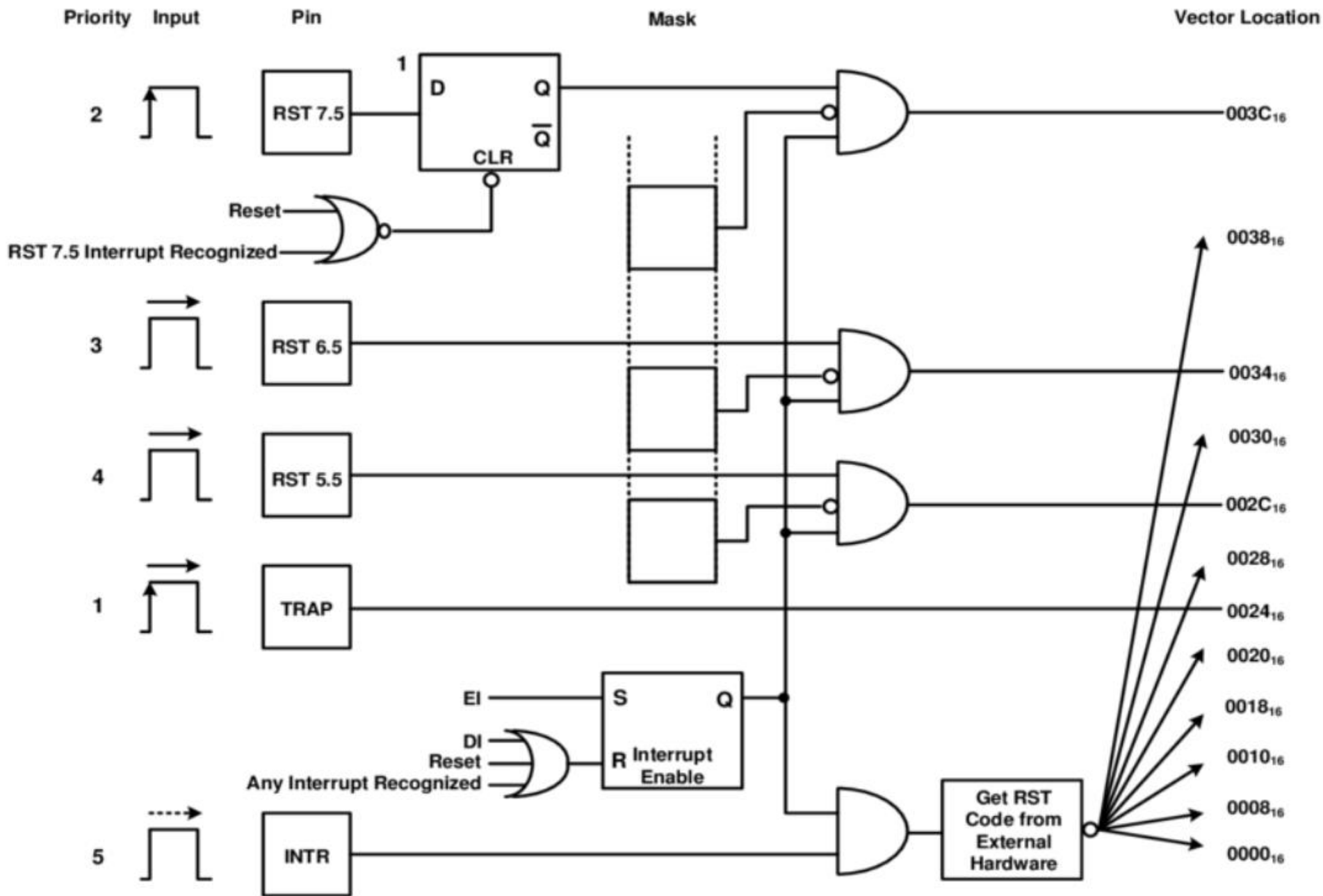


Interrupts in 8085

Interrupts are the signals generated by the external devices to request the microprocessor to perform a task. There are 5 interrupt signals, i.e. TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR.

Interrupt are classified into following groups based on their parameter –

- **Vector interrupt** – In this type of interrupt, the interrupt address is known to the processor. **For example:** RST7.5, RST6.5, RST5.5, TRAP.
- **Non-Vector interrupt** – In this type of interrupt, the interrupt address is not known to the processor so, the interrupt address needs to be sent externally by the device to perform interrupts. **For example:** INTR.
- **Maskable interrupt** – In this type of interrupt, we can disable the interrupt by writing some instructions into the program. **For example:** RST7.5, RST6.5, RST5.5.
- **Non-Maskable interrupt** – In this type of interrupt, we cannot disable the interrupt by writing some instructions into the program. **For example:** TRAP.
- **Software interrupt** – In this type of interrupt, the programmer has to add the instructions into the program to execute the interrupt. There are 8 software interrupts in 8085, i.e. RST0, RST1, RST2, RST3, RST4, RST5, RST6, and RST7.
- **Hardware interrupt** – There are 5 interrupt pins in 8085 used as hardware interrupts, i.e. TRAP, RST7.5, RST6.5, RST5.5, INTA.



Instruction	Opcode	Vector Address
RST 0	C7 H	0000 H
RST 1	CF H	0008 H
RST 2	D7 H	0010 H
RST 3	DF H	0018 H
RST 4	E7 H	0020 H
RST 5	EF H	0028 H
RST 6	F7 H	0030 H
RST 7	FF H	0038 H

To communicate with other devices
two interfacing devices are used

Memory Interfacing
I/o Interfacing

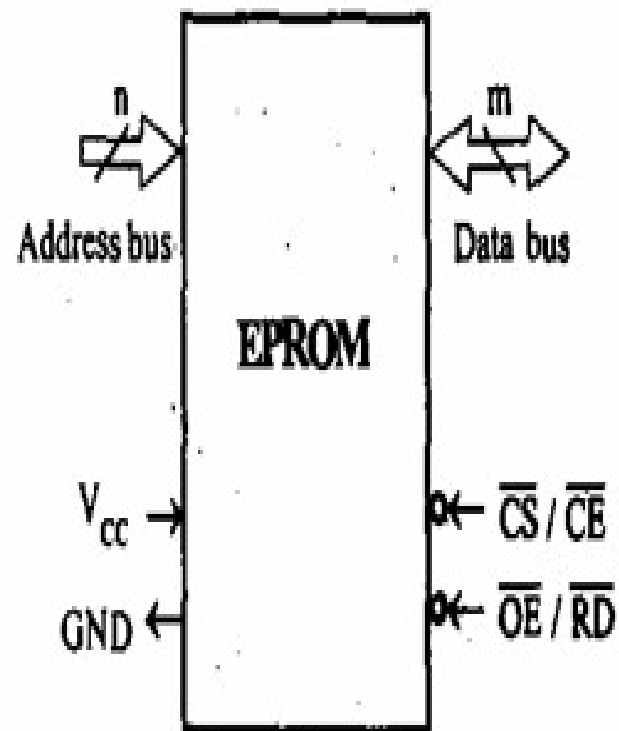
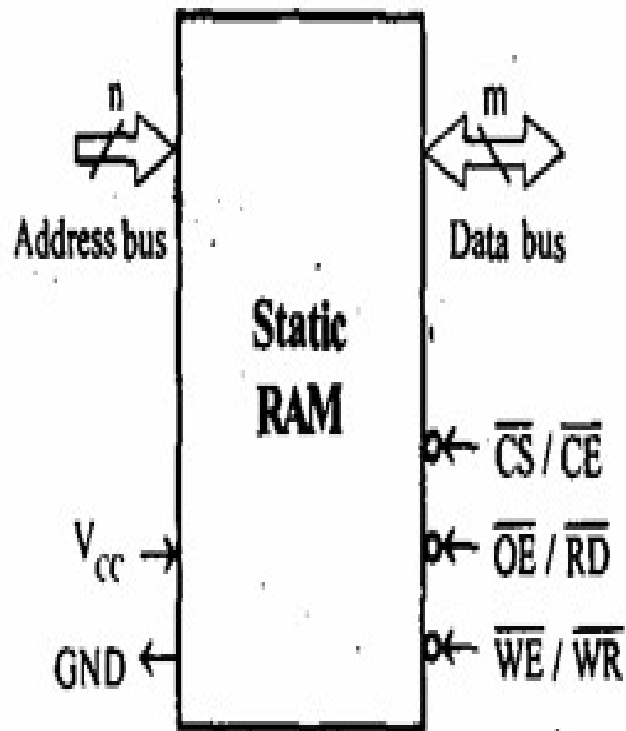
MEMORY INTERFACING

Cpu=8085, processor=8086

16 address lines==2 power 16=64K
Data bus=8 bit

256 x8 MB RAM=> 2 power 8➔address 8,data 8

Device	Function	Internal addresses																				
INTEL 8279	Keyboard/display controller. Used for keyboard scanning and display refreshing.	Two-internal addresses $A_0 = 0 \rightarrow$ Data register $A_0 = 1 \rightarrow$ Control register																				
INTEL 8257 or INTEL 8237	DMA controller. Used for supporting DMA access to I/O device. It acts as a master during DMA mode. It is a slave device during programming mode.	Sixteen-internal addresses <table><tr><td>A_3</td><td>A_2</td><td>A_1</td><td>A_0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>.</td><td>.</td><td>.</td><td>.</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	A_3	A_2	A_1	A_0	0	0	0	0	0	0	0	1	.	.	.	.	1	1	1	1
A_3	A_2	A_1	A_0																			
0	0	0	0																			
0	0	0	1																			
.	.	.	.																			
1	1	1	1																			
INTEL 8259	Interrupt controller. Used to expand the hardware interrupt INTR to eight interrupts in 8085 based system and 256 interrupts in 8086 based system.	Two-internal addresses $A_0 = 0$ $A_0 = 1$																				
INTEL 8253/8254	Programmable Timer. Used in the system to produce various timing signals. It has three independent counters and can be programmed in six operating modes.	Four-internal addresses <table><tr><td></td><td>A_1</td><td>A_0</td></tr><tr><td>Counter-0</td><td>0</td><td>0</td></tr><tr><td>Counter-1</td><td>0</td><td>1</td></tr><tr><td>Counter-2</td><td>1</td><td>0</td></tr><tr><td>Control Register</td><td>1</td><td>1</td></tr></table>		A_1	A_0	Counter-0	0	0	Counter-1	0	1	Counter-2	1	0	Control Register	1	1					
	A_1	A_0																				
Counter-0	0	0																				
Counter-1	0	1																				
Counter-2	1	0																				
Control Register	1	1																				
INTEL 8251 USART	Universal Synchronous/Asynchronous Receiver Transmitter. Used for serial data communication.	Two-internal addresses $C/\overline{D} = 0 \rightarrow$ Data register $C/\overline{D} = 1 \rightarrow$ Control register																				



$\overline{CS} / \overline{CE}$ - Chip select (or Chip enable) ; $\overline{OE} / \overline{RD}$ - Output enable (or Read control)

$\overline{WE} / \overline{WR}$ - Write enable (or Write control)

Dimensions of Memory

- *Memory is usually measured by two numbers: its length and its width (Length X Width).*
 - » *The length is the total number of locations.*
 - » *The width is the number of bits in each location.*
- *The length (total number of locations) is a function of the number of address lines.*

of memory locations = $2^{(\text{# of address lines})}$
- *So, a memory chip with 10 address lines would have*

$2^{10} = 1024$ locations (1K)
- *Looking at it from the other side, a memory chip with 4K locations would need*

$\text{Log}_2 4096 = 12$ address lines

The 8085 and Memory

- *The 8085 has 16 address lines. That means it can address*
 - $2^{16} = 64K$ memory locations.*
 - *Then it will need 1 memory chip with 64 k locations, or 2 chips with 32 K in each, or 4 with 16 K each or 16 of the 4 K chips, etc.*
- *how would we use these address lines to control the multiple chips?*

Memory IC EPROM/RAM	Capacity	Number of address pins	Number of data pins
2708/6208	1kb	10	8
2716/6216	2kb	11	8
2732/6232	4kb	12	8
2764/6264	8kb	13	8
27256/62256	32kb	15	8
27512/62512	64kb	16	8
27010/62128	128kb	17	8
27020/62138	256kb	18	8
27040/62148	512kb	19	8

Memory interfacing techniques

Absolute Decoding / Full Decoding

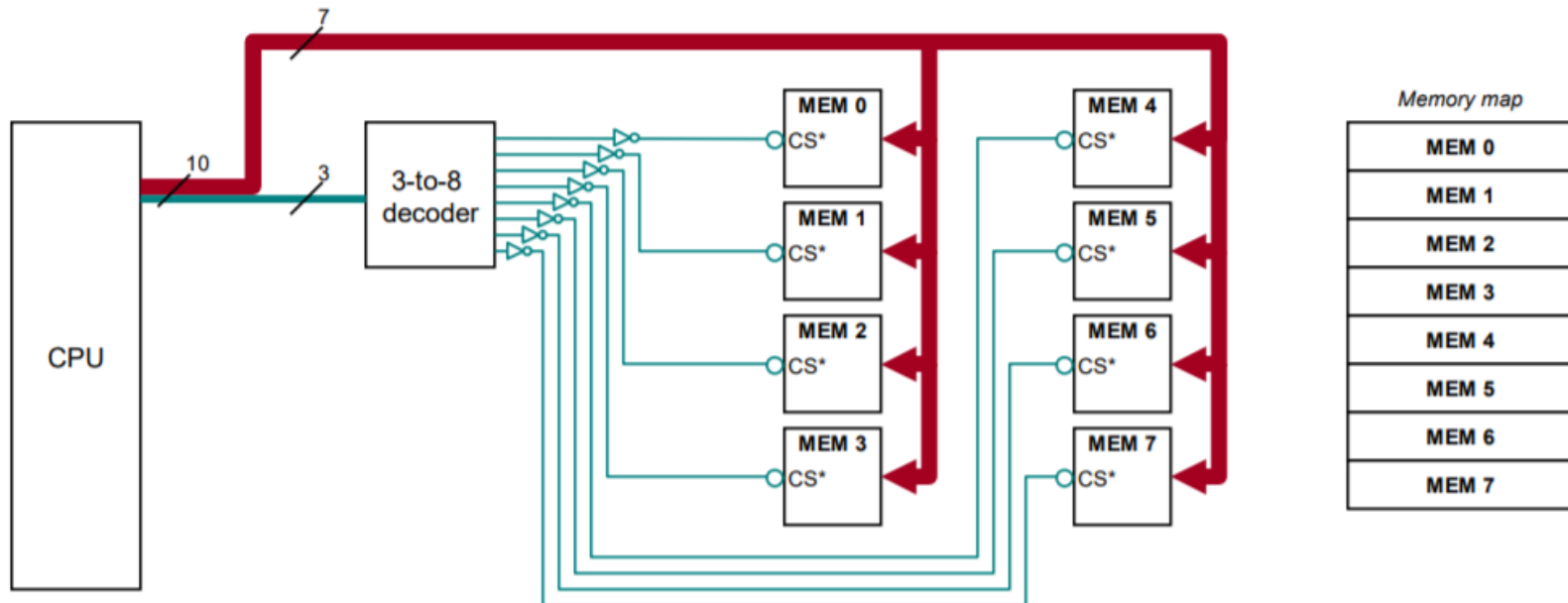
Partial decoding

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			0	0	0	x	x	x	x	x	x	x	X	x	x
			0	0	1	X									
			0	1	0	X									
			0	1	0	x									

4 kb =10 address

A very simple example

- Let's assume a very simple microprocessor with 10 address lines (1KB memory)
- Let's assume we wish to implement all its memory space and we use 128x8 memory chips
- **SOLUTION**
 - We will need 8 memory chips ($8 \times 128 = 1024$)
 - We will need 3 address lines to select each one of the 8 chips
 - Each chip will need 7 address lines to address its internal memory cells

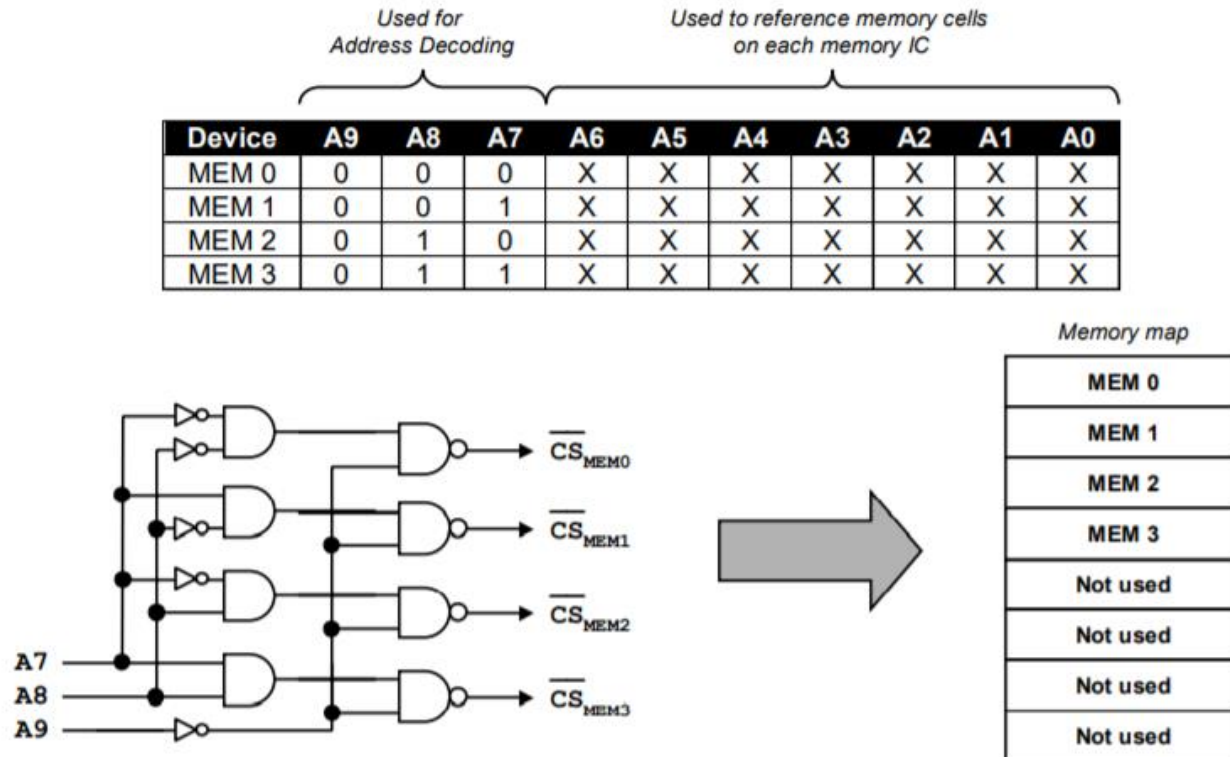


Full address decoding

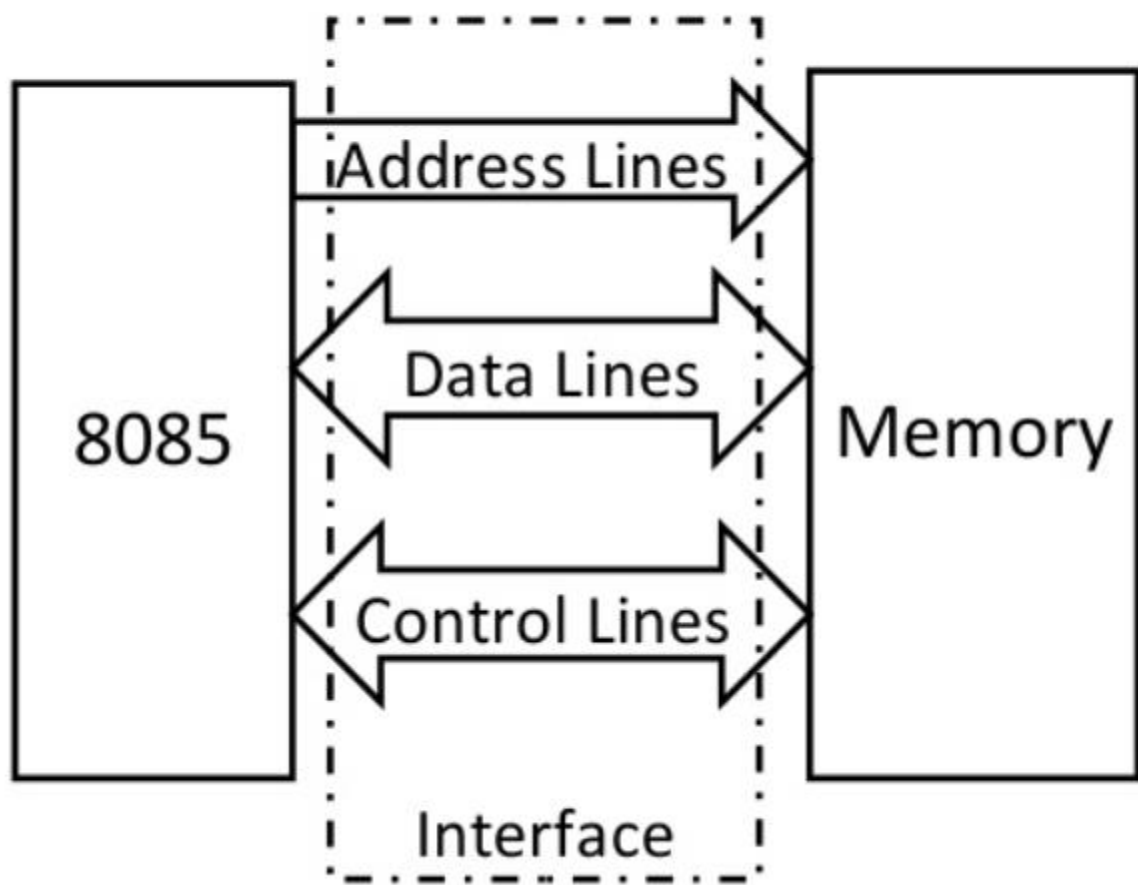
■ Let's assume the same microprocessor with 10 address lines (1KB memory)

- However, this time we wish to implement only 512 bytes of memory
- We still must use 128-byte memory chips
- Physical memory must be placed on the upper half of the memory map

■ SOLUTION

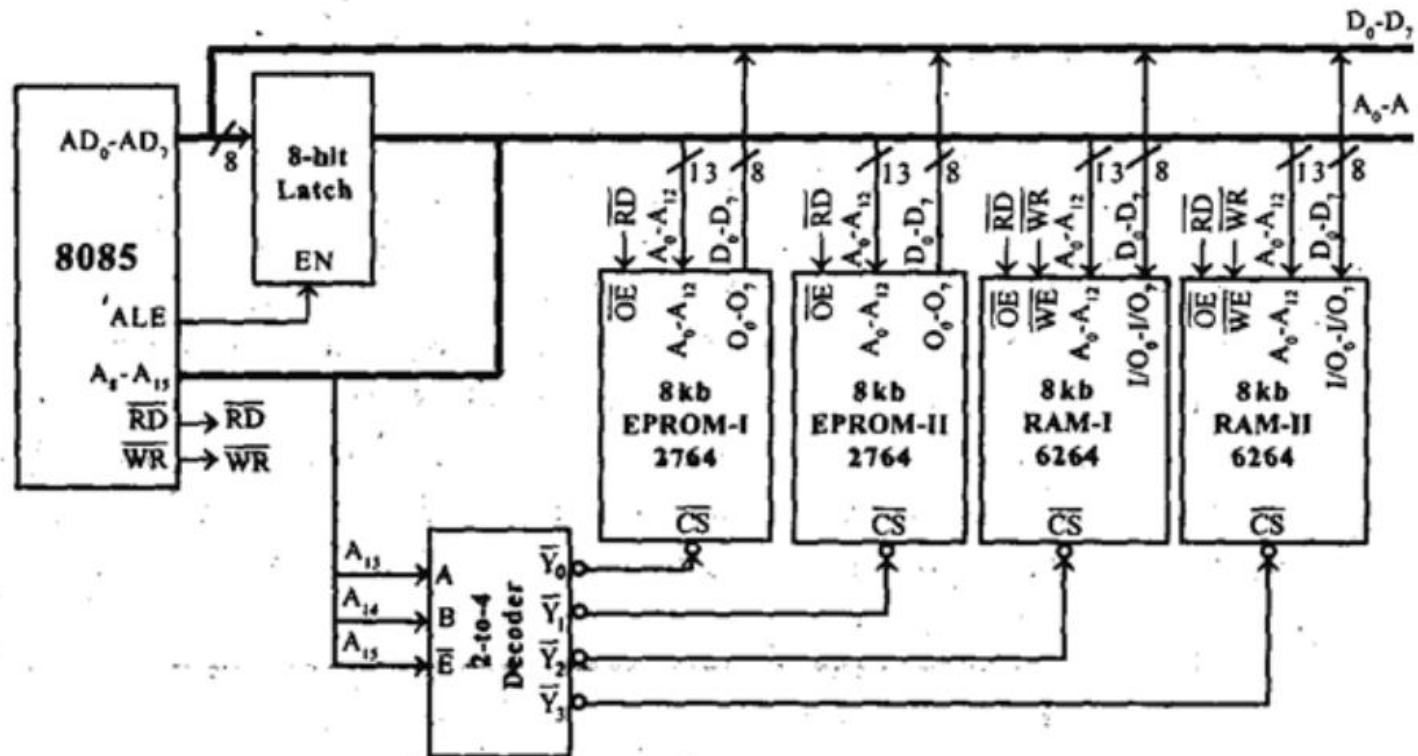


Example Block Diagram



Consider a system in which 32kb memory space is implemented using four numbers of 8kb memory. Interface the EPROM and RAM with 8085 processor.

- The total memory capacity is 32Kb. So, let two number of 8kb n memory be EPROM and the remaining two numbers be RAM.
- Each 8kb memory requires 13 address lines and so the address lines A0- A12 of the processor are connected to 13 address pins of all the memory.
- The address lines and A13 - A14 can be decoded using a 2-to-4 decoder to generate four chip select signals.
- These four chip select signals can be used to select one of the four memory IC at any one time.
- The address line A15 is used as enable for decoder.
- The simplified schematic memory organization is shown.



Consider a system in which the 64kb memory space is implemented using eight numbers of 8kb memory. Interface the EPROM and RAM with 8085 processor.

- The total memory capacity is 64Kb. So, let 4 numbers of 8Kb EPROM and 4 numbers of 8Kb RAM.
- Each 8kb memory requires 13 address lines. So the address line A0 - A12 of the processor are connected to 13address pins of all the memory ICs.
- The address lines A13, A14 and A15 are decoded using a 3-to-8 coder to generate eight chip select signals. These eight chip select signals can be used to select one of the eight memories at any one time.
- The memory interfacing is shown in following figure.

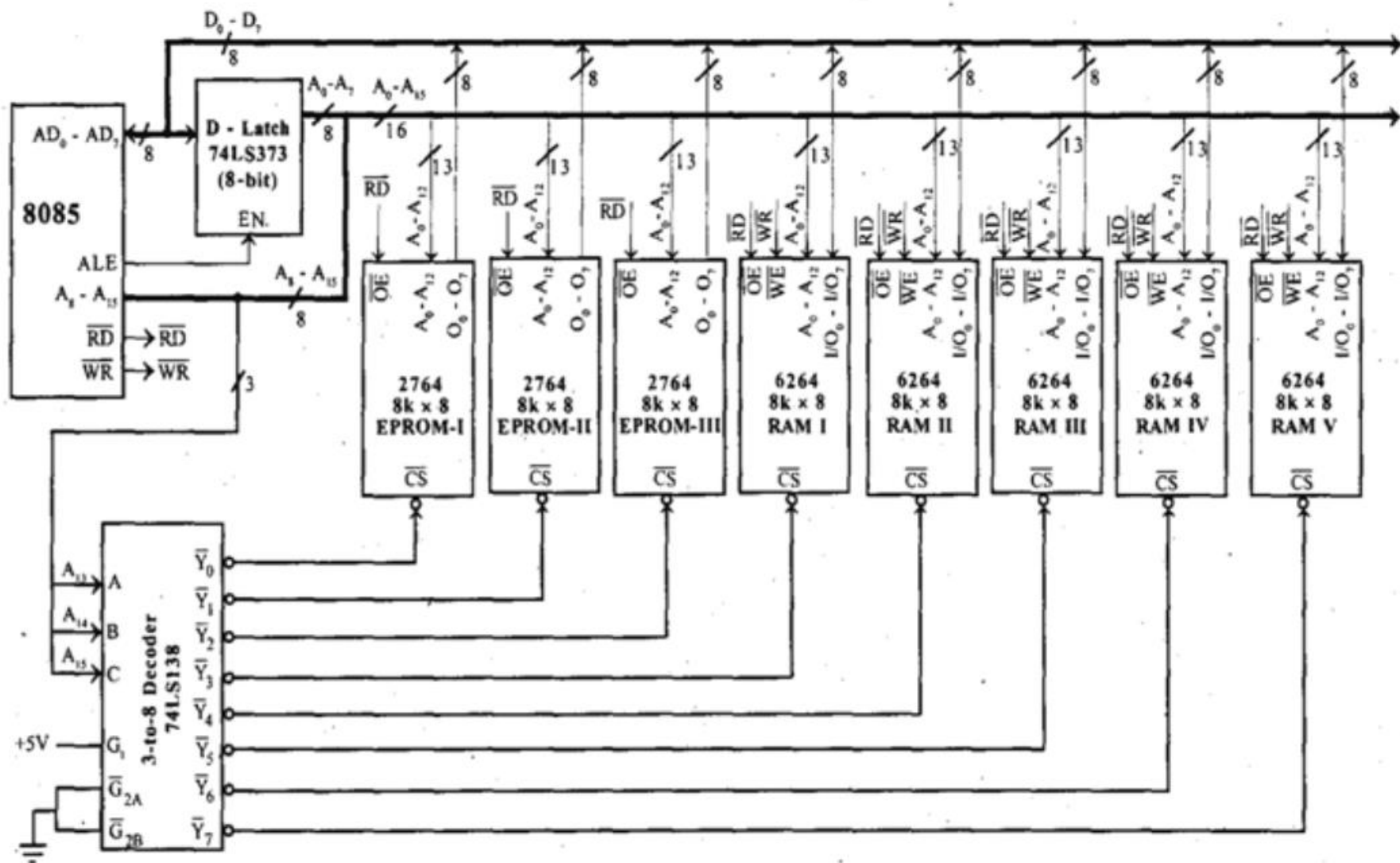


Fig - Interfacing 4 no. 8Kb EPROM and 4 no. 8Kb RAM with 8085

[illegible]

I/O Intefacing

- *I/O devices can be interfaced to an 8085*

I/O Mapped I/O

Memory Mapped I/O

I/O Mapped I/O

- ***IN addr8***

the content of port is moved to A- Register

- ***OUT addr8***

The content of A register is moved to port

Memory Mapped I/O

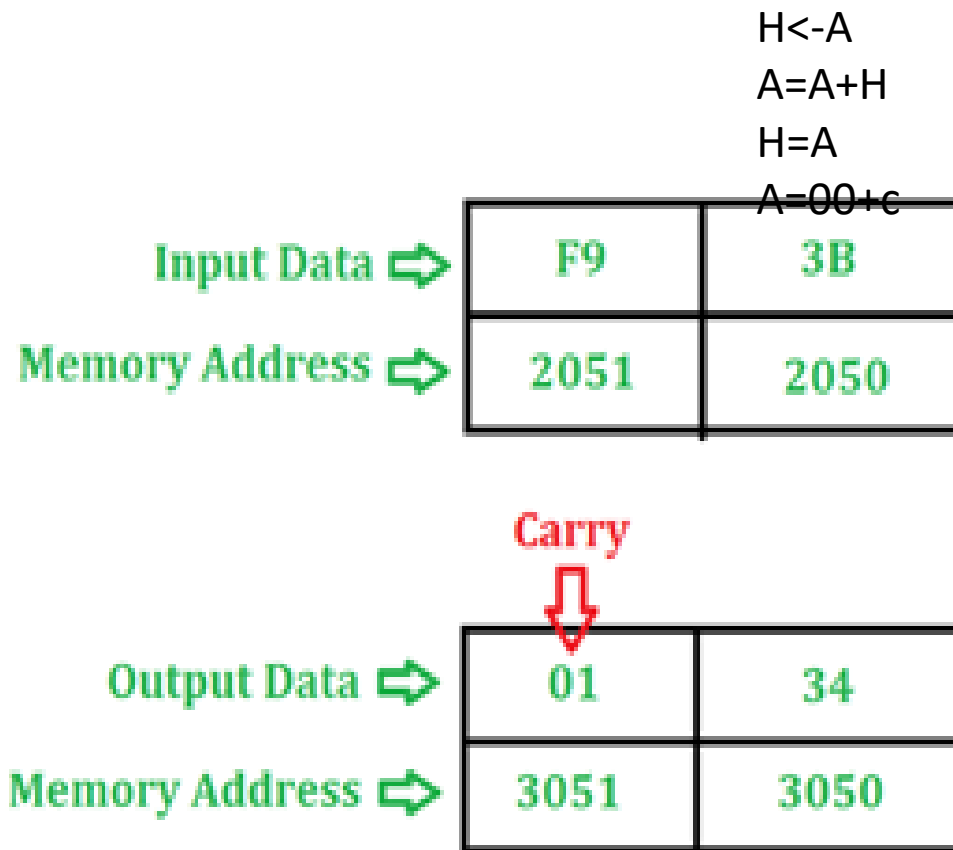
- *Mov r,m* *LDA addr*
- *LHLD addr* *ADD M*
- *XRAM M* *MOV m,r*
- *STA addr*

Comparison of memory mapped I/O and Peripheral I/O

Characteristics	Memory Mapped	I/O Mapped
Device address	16 bit	8 bit
Control signals for I/O	MEMW MEMR	IOR IOW
Instruction available	STA, LDA, LDAX, STAX etc.,	IN & OUT
Data Transfer	between any register and I/O	Between I/O and accumulator
Maximum no of I/O	64k	Independent
Execution speed	13T for STA &LDA	10T
Hardware	More hardware require	Less hardware

Problem – Write an assembly language program to add two 8 bit numbers stored at address 2050 and address 2051 in 8085 microprocessor. The starting address of the program is taken as 2000.

Example –



F9 +3B

1	1	1	1	1	0	0	1
0	0	1	1	1	0	1	1
0	0	1	1	0	1	0	0
1							

01 34

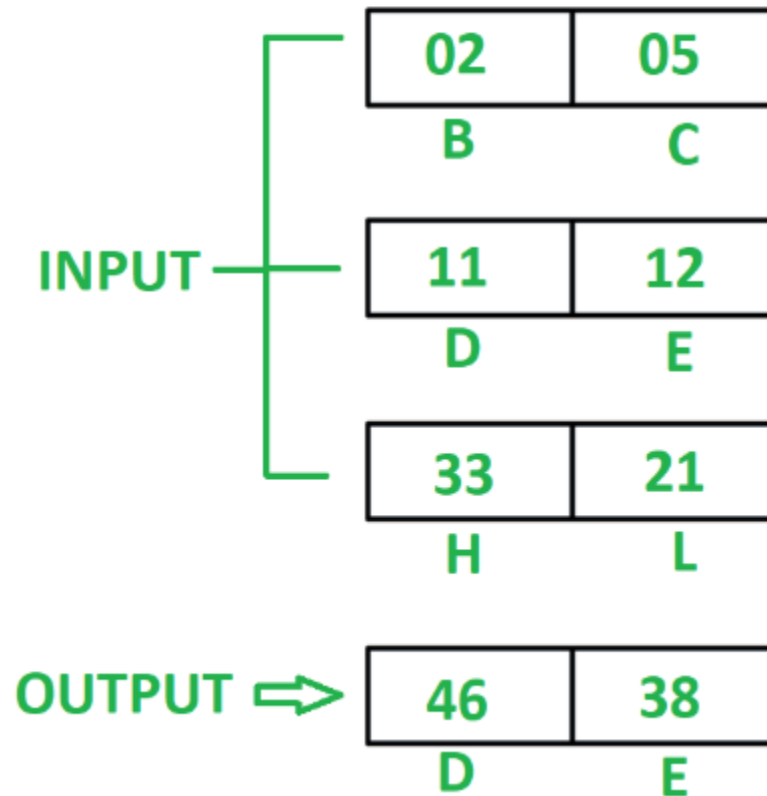
Algorithm –

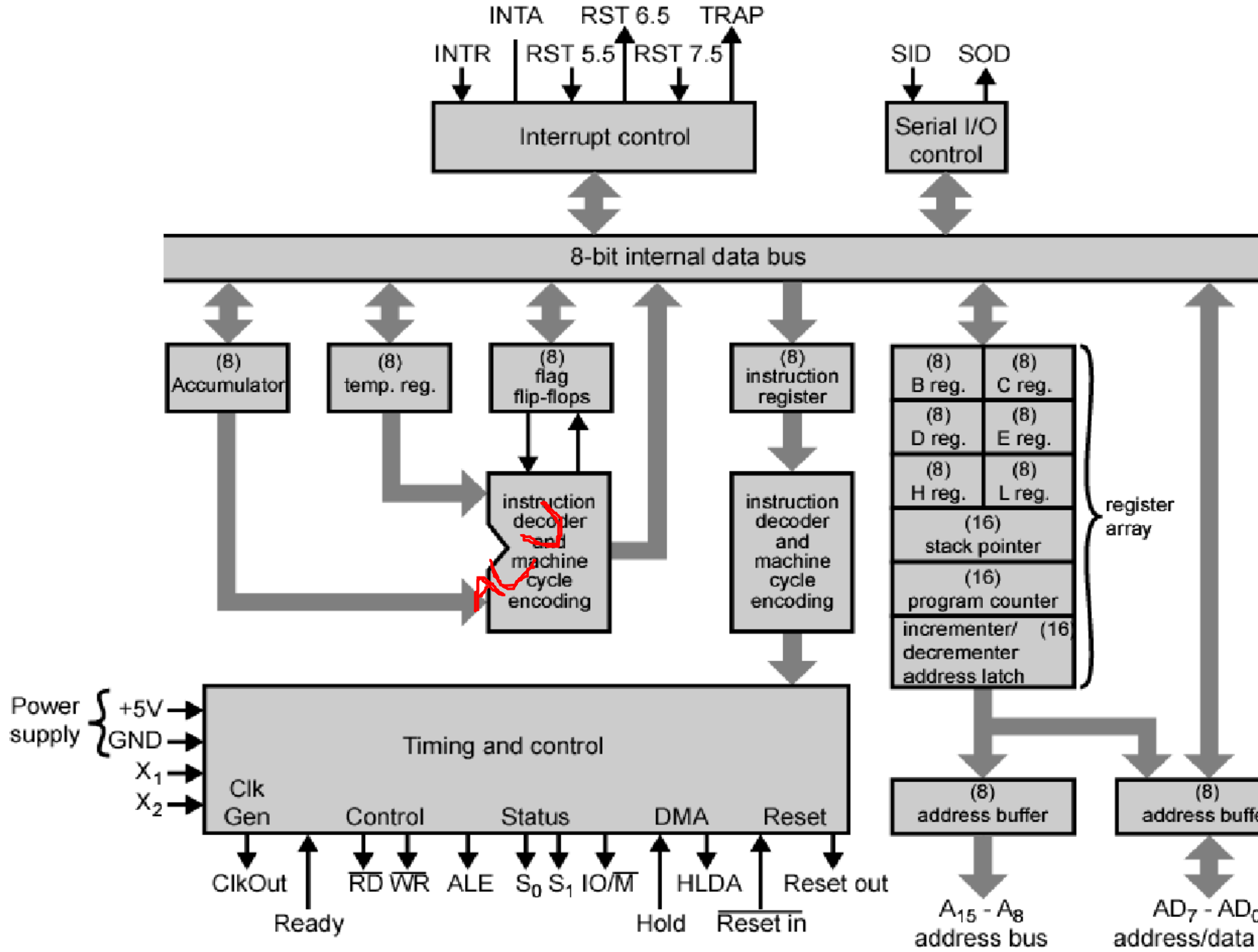
1. Load the first number from memory location 2050 to accumulator.
2. Move the content of accumulator to register H.
3. Load the second number from memory location 2051 to accumulator.
4. Then add the content of register H and accumulator using “ADD” instruction and storing result at 3050
5. The carry generated is recovered using “ADC” command and is stored at memory location 3051

MEMORY ADDRESS	MNEMONICS	COMMENT
2000	LDA 2050	$A \leftarrow [2050]$
2003	MOV H, A	$H \leftarrow A$
2004	LDA 2051	$A \leftarrow [2051]$
2007	ADD H	$A \leftarrow A + H$
2006	MOV L, A	$L \leftarrow A$
2007	MVI A 00	$A \leftarrow 00$
2009	ADC A	$A \leftarrow A + A + \text{carry}$
200A	MOV H, A	$H \leftarrow A$
200B	SHLD 3050	$H \rightarrow 3051, L \rightarrow 3050$
200E	HLT	

Problem – Write an assembly language program to add three 16 bit numbers stored in register HL, DE, BC and store the result in DE with minimum number of instructions.

Example –





2000	DAD D	$H \leftarrow H + D, L \leftarrow L + E$
2001	MOV D, B	$D \leftarrow B$
2002	MOV E, C	$E \leftarrow C$
2003	DAD D	$H \leftarrow H + D, L \leftarrow L + E$
2004	XCHG	Swap content of HL with DE
2005	HLT	END

Problem – Multiply two 8 bit numbers stored at address 2050 and 2051. Result is stored at address 3050 and 3051. Starting address of program is taken as 2000.

Example –

Input Data ➡	07	43
Memory Address ➡	2051	2050

Output Data ➡	01	D5
Memory Address ➡	3051	3050

			1	1
4	3		0	1
4	3		1	1
4	3	0	0	x
4	3	0	1	1
4	3			

For Reference purpose

Mohd. Ali mahizi (microprocessor and microcontroller)

B.Ram(interrupt)

Geeks for geeks (8085 assembly programming)

MEMORY ADDRESS	MNEMONICS	COMMENT
2000	LHLD 2050	$H \leftarrow 2051, L \leftarrow 2050$
2003	XCHG	$H \leftrightarrow D, L \leftrightarrow E$
2004	MOV C, D	$C \leftarrow D$
2005	MVI D 00	$D \leftarrow 00$
2007	LXI H 0000	$H \leftarrow 00, L \leftarrow 00$
200A	DAD D	$HL \leftarrow HL + DE$
200B	DCR C	$C \leftarrow C - 1$
200C	JNZ 200A	If Zero Flag=0, goto 200A
200F	SHLD 3050	$H \rightarrow 3051, L \rightarrow 3050$
2012	HLT	

H=07,L=43

D=07,E=43

C=07

D=0

HL=0000

Loop“:

$HL = 0000 + 0043 = 0043$ --- $c = 7 - 1 = 6$

C not equal to zero

$HL = 0043 + 0043 = 0086$ --- $c = 6 - 1 = 5$

c

Last Year Papers

- Q1 (a) Write an assembly language program to move a block of data from the location 5000H to the location 8000H using 8085 microprocessor (for block of 6 byte) (3)
- (b) Which addressing mode in 8085 moves a byte (or a word) between a memory location and a register? Also give an example of this type of addressing. (3)

- Q2 (a) What are the different programmable registers in 8085 microprocessor? Explain all. (5)
- (b) Explain the need to demultiplex the bus AD₇-AD₀. (3)
- (c) What are the different vectored interrupts in 8085 microprocessor? Briefly explain. (4.5)
- 

Last year paper

Q1. Attempt all parts:

(2.5x10=25)

- a) Explain $IO/\overline{M}$ ALE pin of 8085 Microprocessor.
- b) What are flag registers? Explain auxiliary carry and parity flag of 8085 microprocessor.

Q2. a) Write a program to find the square root of a number. Also explain it with the help of suitable flow chart.

(7.5)

- b) Write a program to generate Fibonacci series.

(5)

Q3. a) Draw and explain hardware interrupt structure 8085 microprocessor.

(7.5)

Interrupts in 8085 microprocessor

Last Updated: 09-09-2019

When microprocessor receives any interrupt signal from peripheral(s) which are requesting its services, it stops its current execution and program control is transferred to a sub-routine by generating **CALL** signal and after executing sub-routine by generating **RET** signal again program control is transferred to main program from where it had stopped.

When microprocessor receives interrupt signals, it sends an acknowledgement (INTA) to the peripheral which is requesting for its service.

Interrupts can be classified into various categories based on different parameters:

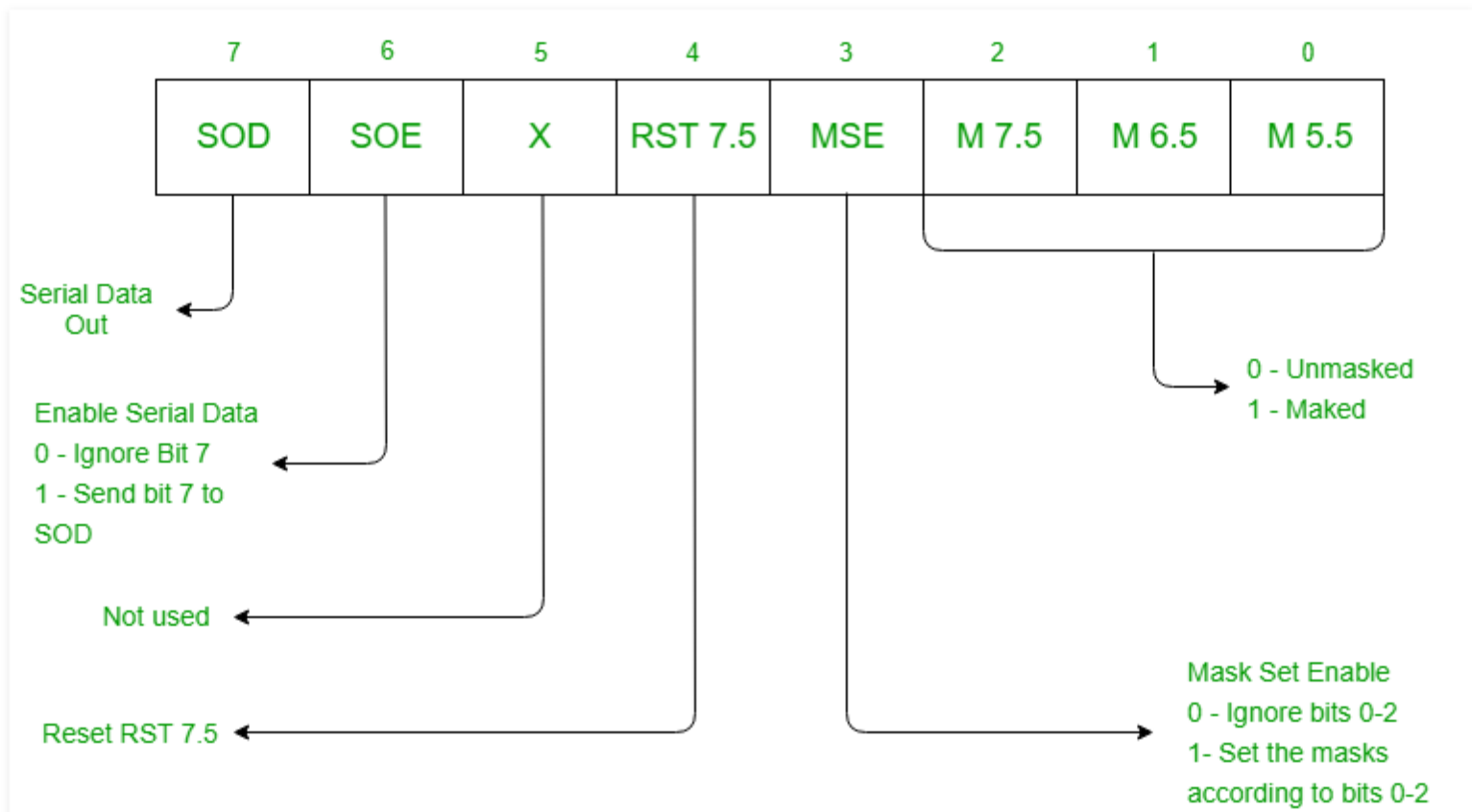
1. **Hardware and Software Interrupts –**

When microprocessors receive interrupt signals through pins (hardware) of microprocessor, they are known as *Hardware Interrupts*. There are 5 Hardware Interrupts in 8085 microprocessor. They are – *INTR, RST 7.5, RST 6.5, RST 5.5, TRAP*

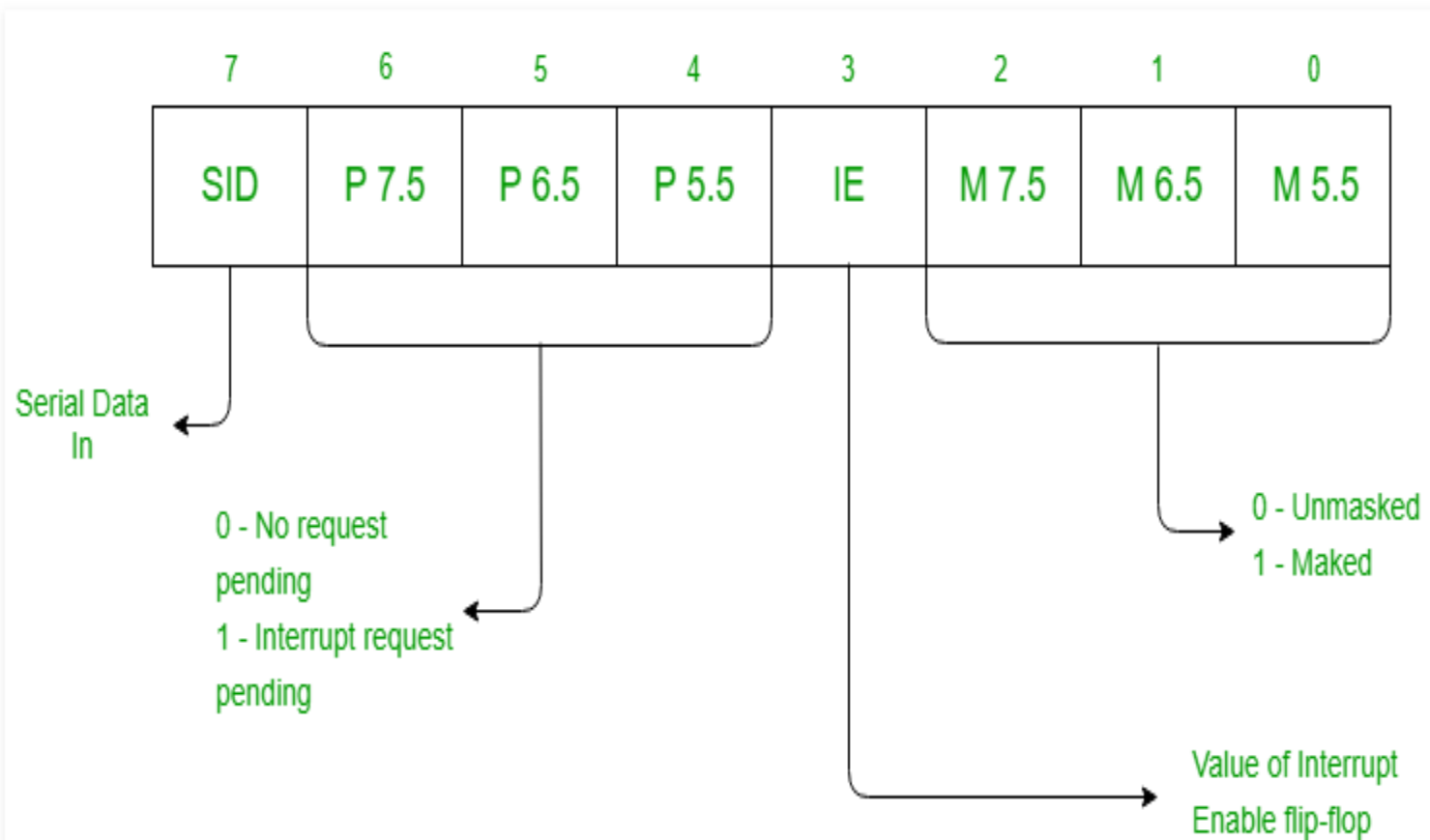
Software Interrupts are those which are inserted in between the program which means these are mnemonics of microprocessor. There are 8 software interrupts in 8085 microprocessor. They are – *RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7*.

Instruction for Interrupts –

1. **Enable Interrupt (EI)** – The interrupt enable flip-flop is set and all interrupts are enabled following the execution of next instruction followed by EI. No flags are affected. After a system reset, the interrupt enable flip-flop is reset, thus disabling the interrupts. This instruction is necessary to enable the interrupts again (except TRAP).
2. **Disable Interrupt (DI)** – This instruction is used to reset the value of enable flip-flop hence disabling all the interrupts. No flags are affected by this instruction.
3. **Set Interrupt Mask (SIM)** – It is used to implement the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by setting various bits to form masks or generate output data via the Serial Output Data (SOD) line. First the required value is loaded in accumulator then SIM will take the bit pattern from it.



4. **Read Interrupt Mask (RIM)** – This instruction is used to read the status of the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by loading into the A register a byte which defines the condition of the mask bits for the interrupts. It also reads the condition of SID (Serial Input Data) bit on the microprocessor.



Problem – Write an assembly language program for calculating the factorial of a number using 8086 microprocessor

Examples –

Input : 04H

Output : 18H

as In Decimal : $4*3*2*1 = 24$

In Hexadecimal : $24 = 18H$

Input : 06H

Output : 02D0H

as In Decimal : $6*5*4*3*2*1 = 720$

In Hexadecimal : $720 = 02D0H$

Assumptions –

Starting address of program: 0400

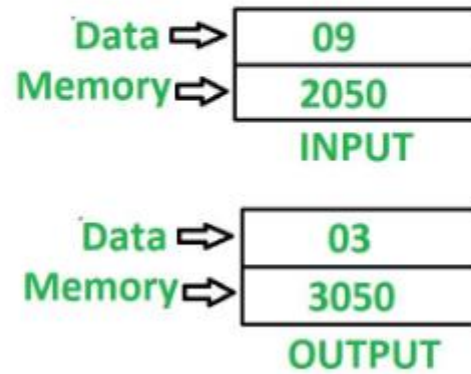
Input memory location: 0500

Output memory location: 0600 and 0601

ADDRESS	LABEL	MNEMONIC	COMMENT
2000H	Data		Data Byte
2001H	Result		Result of factorial
2002H		LXI H, 2000H	Load data from memory
2005H		MOV B, M	Load data to B register
2006H		MVI D, 01H	Set D register with 1
2008H	FACTORIAL	CALL MULTIPLY	Subroutine call for multiplication
200BH		DCR B	Decrement B
200CH		JNZ FACTORIAL	Call factorial till B becomes 0
200FH		INX H	Increment memory
2010H		MOV M, D	Store result in memory
2011H		HLT	Halt
2100H	MULTIPLY	MOV E, B	Transfer contents of B to C
2101H		MVI A, 00H	Clear accumulator to store result
2103H	MULTIPLYLOOP	ADD D	Add contents of D to A
2104H		DCR E	Decrement E
2105H		JNZ MULTIPLYLOOP	Repeated addition
2108H		MOV D, A	Transfer contents of A to D
2109H		RET	Return from subroutine

Problem – Write an assembly language program in 8085 microprocessor to find square root of a number.

Example –



MEMORY ADDRESS	MNEMONICS	COMMENT
2000	MVI D, 01	D <- 01
2002	MVI E, 01	E <- 01
2004	LDA 2050	A <- M[2050]
2007	SUB D	A <- A - D
2008	JZ 2011	Jump if ZF = 0 to memory location 2011
200B	INR D	D <- D + 1
200C	INR D	D <- D + 1
200D	INR E	E <- E + 1
200E	JMP 2007	Jump to memory location 2007
2011	MOV A, E	A <- E
2012	STA 3050	A -> M[3050]
2015	HLT	END